



SWINBURNE UNIVERSITY OF TECHNOLOGY

SWE40006 - Software Deployment and Evolution

Deployment Portfolio Task 4

Container Application Deployment & Orchestration using Docker

DECLARED TARGET LEVEL: HIGH DISTINCTION (Task 4.4)

Field	Detail
Student Name	Karamjot Singh
Student ID	104809887
Unit Code & Title	SWE40006 - Software Deployment and Evolution
Semester	Semester 2, 2026
Assessment	Deployment Portfolio Task 4
Submission Date	27/09/2026
Declared Target Level	High Distinction - Tasks 4.1, 4.2, 4.3 and 4.4 all completed
Docker Hosts	Host 1: Windows 11, Docker Desktop (WSL 2) Host 2: AWS EC2, Amazon Linux 2023, Docker Engine (ap-southeast-2)
Docker Hub Account	https://hub.docker.com/u/Karamjot1008
Source Repository	https://github.com/Karam1008/swe40006-task4-docker

Table of Contents

1. Introduction and Scope.....	4
1.1 Purpose of the Task.....	4
1.2 Naming Convention	4
1.3 Live Verification Endpoints	5
2. Architecture Overview	6
2.1 Final Deployment Topology	6
3. Task 4.1 - Pass Level: Docker Environment Setup	8
3.1 WSL 2 Installation.....	8
3.2 Docker Desktop Installation	9
3.3 Docker Hub Account and Sign-in	11
3.4 Verification.....	11
3.5 Limits of the Pass-Level Result.....	13
4. Task 4.2 - Credit Level: Containerised Python Application on Two Hosts	14
4.1 Source Control and Registry Preparation	14
4.2 Application Design	15
4.3 Dockerfile	16
4.4 Build	16
4.5 Local Deployment on a Designated Port.....	17
4.6 Publishing to Docker Hub.....	18
4.7 Secondary Docker Host Provisioning	20
4.8 Multi-Host Verification	22
4.9 Weaknesses of the Credit-Level Design	23
5. Task 4.3 - Distinction Level: Custom Web Application with an Optimised Build	24
5.1 Application Design	24
5.2 Service Architecture.....	24
5.3 Optimised Dockerfile	25
5.4 Measured Effect of the Optimisation	25
5.5 Local Deployment and Health Checks.....	27
5.6 Container Networking.....	29
5.7 Environment Variable Configuration	30
5.8 Persistence Across Container Recreation	31
5.9 Publishing to Docker Hub.....	31
5.10 Public Deployment on the Secondary Host	32
5.11 HTTPS with Automatic Certificates	32
6. Task 4.4 - High Distinction Level: Containerised Non-Web Application	34
6.1 Tool Design.....	34

6.2 Dockerfile and Container Best Practices.....	35
6.3 Tests at Build Time	35
6.4 Build and Interface.....	36
6.5 Storage Design	37
6.6 One-Shot Execution in a Hardened Container	38
6.7 Exit Codes.....	38
6.8 State Beyond the Container Lifetime.....	39
6.9 Long-Running Lifecycle and Graceful Shutdown	40
6.10 Portability to the Secondary Host.....	41
6.11 Practice Across the Four Levels.....	42
7. Self-Troubleshooting Log and Analysis	43
7.1 Bind-Mount Permission Denied on the Linux Host.....	43
7.2 Reverse Proxy Restart Loop Caused by an Invalid Site Address	43
7.3 Host Port 80 Unavailable on the Windows Host.....	44
7.4 HTTPS URL refused connections.	44
8. Limitations and Future Work	45
8.1 Single-Host Orchestration.....	45
8.2 Stateful Service Without Replication or Backup	45
8.3 Manual Build and Publish	45
8.4 No Image Scanning, Signing or Digest Pinning.....	45
8.5 HTTPS Relies on a Shared Public Domain	45
8.6 Registry Credentials and Private Images.....	45
8.7 Scope of the Log Analysis Tool.....	45
9. External Code and Script Repository.....	46
10. Generative AI Declaration.....	47
10.1 Tool Used and Scope of Use.....	47
10.2 What Was Performed by Me	47
10.3 Verification and Independent Judgement	47
10.4 Statement	47
11. References	49

1. Introduction and Scope

This report documents the containerisation, packaging, publication and deployment of three applications using Docker, completed for Deployment Portfolio Task 4 of SWE40006. All four grade bands were attempted and completed in sequence: 4.1 (Pass), 4.2 (Credit), 4.3 (Distinction) and 4.4 (High Distinction).

Two Docker hosts were used throughout. Host 1 is my Windows 11 laptop running Docker Desktop on the WSL 2 backend, where every image was built and tested. Host 2 is an Amazon EC2 instance running Docker Engine on Amazon Linux 2023 in Asia Pacific (Sydney), which pulls the published images from Docker Hub and serves them publicly. No image is ever built on Host 2: it runs only what the registry provides, which is what makes it a genuine test of portability rather than a second build environment.

1.1 Purpose of the Task

The task is structured as a progression. Each grade band deliberately removes a weakness left by the previous one:

- **Pass (4.1)** establishes a working container runtime and a registry account, verified with the official hello-world image. It proves the toolchain works but deploys nothing of my own.
- **Credit (4.2)** produces the first custom image, a basic Flask application, publishes it to Docker Hub and runs the identical artefact on a second host. The Dockerfile is intentionally simple, and its weaknesses are recorded at the end of that section.
- **Distinction (4.3)** removes those weaknesses with an optimised multi-stage Dockerfile, and moves from one container to a three-service application with isolated networks, environment-driven configuration, health checks, persistent storage and public HTTP/HTTPS access.
- **High Distinction (4.4)** applies the same practices to a non-web workload, a command-line log analysis tool, where the concerns shift from ports to volumes, exit codes, signal handling and the container lifecycle.

1.2 Naming Convention

A consistent `swe40006-` prefix was applied to every image and cloud resource so that all artefacts could be identified unambiguously during verification and teardown.

Resource	Identifier
GitHub repository	<code>swe40006-task4-docker</code>
Credit image (4.2)	<code>Karamjot1008/swe40006-flask-basic:1.0.0</code>
Distinction image (4.3)	<code>Karamjot1008/swe40006-docknotes:1.0.0</code>
High Distinction image (4.4)	<code>Karamjot1008/swe40006-logalyzer:1.0.0</code>
Compose projects	<code>docknotes</code> , <code>logalyzer</code>
Docker networks	<code>docknotes_frontend</code> , <code>docknotes_backend</code>
Docker volumes	<code>docknotes_redis_data</code> , <code>docknotes_caddy_data</code> , <code>logalyzer_state</code>

Resource	Identifier
Secondary Docker host (EC2)	SWE40006-DockerHost
Security group / IAM role	swe40006-docker-host-sg / swe40006-docker-host-role

1.3 Live Verification Endpoints

The following endpoints were live at the time of submission and may be used for grading verification.

Endpoint	Value
Credit Flask app (Host 2)	http://52.62.216.119:8080/
Credit health check	http://52.62.216.119:8080/health
DockNotes (HTTP)	http://52.62.216.119/
DockNotes (HTTPS)	https://52.62.216.119.sslip.io/
DockNotes status API	http://52.62.216.119/api/info
Docker Hub repositories	https://hub.docker.com/u/karamjot1008
Source repository	https://github.com/Karam1008/swe40006-task4-docker

The High Distinction application is a command-line tool with no network listener, so it has no URL. It can be verified from any Docker host by pulling the public image:

```
docker run --rm karamjot1008/swe40006-logalyzer:1.0.0 --help.
```

2. Architecture Overview

The deployment evolved across four distinct states. The table below summarises that evolution; each row is expanded in the section that follows.

Level	Artefact	Runtime & Networking	Configuration & State
4.1 Pass	Official hello-world image	Docker Desktop, WSL 2 backend; container runs once and exits	None
4.2 Credit	Custom Flask image, single-stage Dockerfile, pushed to Docker Hub	One container on each host; host port 8080 mapped to container port 5000	PORT and DEPLOY_HOST environment variables; stateless
4.3 Distinction	Custom FastAPI image, optimised multi-stage Dockerfile	Three containers via Compose; only the Caddy proxy publishes ports; internal backend network	.env-driven configuration; Redis data on a named volume; automatic HTTPS
4.4 High Distinction	Standard-library CLI image with a build-time test stage	No network listener; one-shot and long-running modes	Bind mounts for input/output; named volume for run history; exit codes and SIGTERM handling

2.1 Final Deployment Topology

Images are built on Host 1, published to Docker Hub, and pulled by Host 2. GitHub carries the source and the two small configuration files that Host 2 needs to start the Distinction stack:

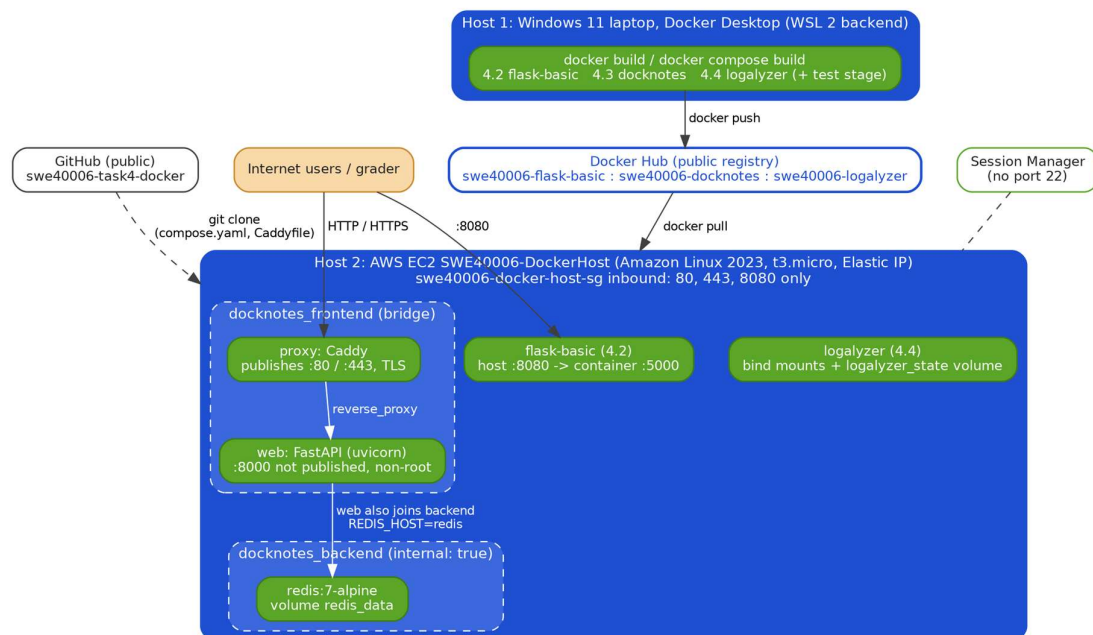


Figure 1: Deployment topology across the two Docker hosts, the registry and the source repository

Two properties of this topology are:

- **Images move between hosts only through the registry.** Host 2 builds nothing; every container it runs is pulled from Docker Hub and is byte-identical to what was tested on Host 1, which is proven by digest comparison in Section 4.8.
- **Minimal exposure on Host 2.** The only listeners reachable from the internet are the Caddy proxy (80/443) and the Credit application (8080). The DockNotes web container and Redis publish no ports, and there is no SSH port; administrative access is through AWS Systems Manager Session Manager.

3. Task 4.1 - Pass Level: Docker Environment Setup

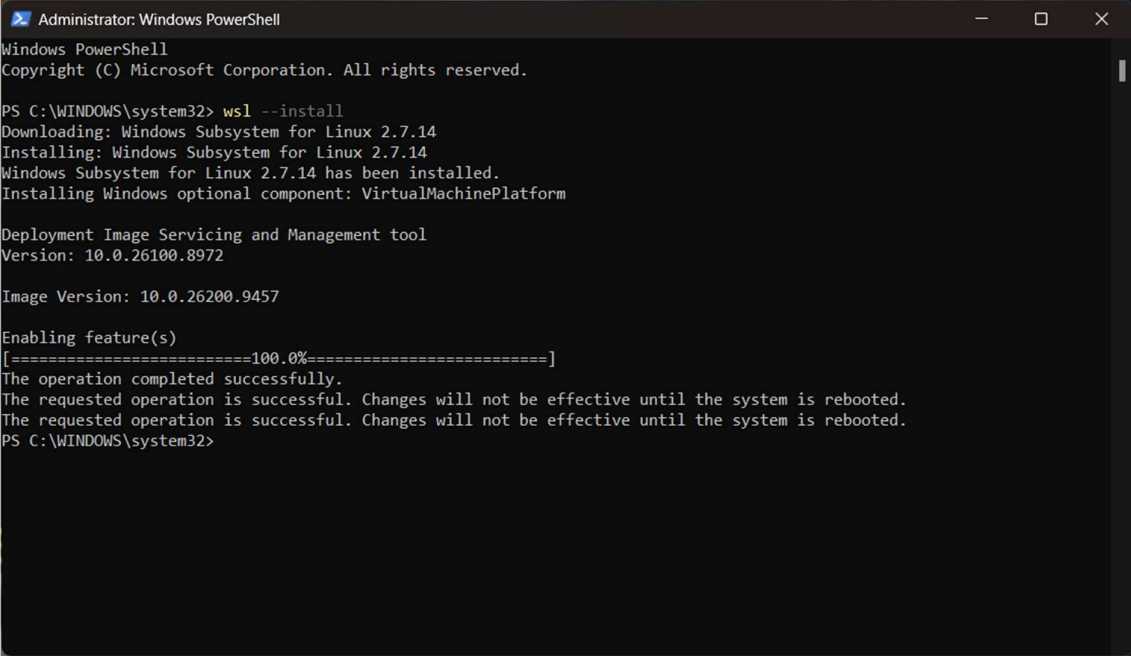
Objective: register an active Docker Hub account, install Docker Desktop on the host machine, and pull and run the official hello-world image to confirm the runtime environment is operational.

3.1 WSL 2 Installation

Docker Desktop on Windows runs the Docker Engine inside a lightweight Linux virtual machine. The WSL 2 backend provides that virtual machine through the Windows Subsystem for Linux, rather than through a Hyper-V machine managed by Docker itself. WSL 2 was installed from an elevated PowerShell session:

```
# PowerShell, Run as Administrator
wsl --install

# after restarting the machine
wsl --version
```



```
Administrator: Windows PowerShell
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\WINDOWS\system32> wsl --install
Downloading: Windows Subsystem for Linux 2.7.14
Installing: Windows Subsystem for Linux 2.7.14
Windows Subsystem for Linux 2.7.14 has been installed.
Installing Windows optional component: VirtualMachinePlatform

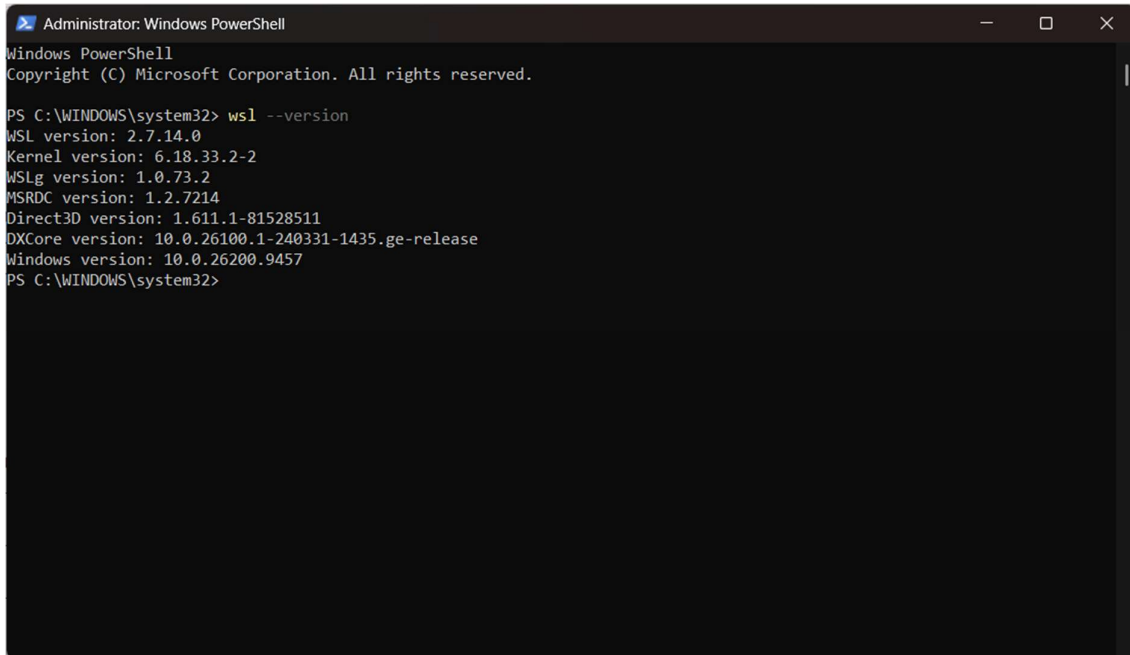
Deployment Image Servicing and Management tool
Version: 10.0.26100.8972

Image Version: 10.0.26200.9457

Enabling feature(s)
[=====100.0%=====]
The operation completed successfully.
The requested operation is successful. Changes will not be effective until the system is rebooted.
The requested operation is successful. Changes will not be effective until the system is rebooted.
PS C:\WINDOWS\system32>
```

Figure 2: `wsl --install` executed from an Admin PowerShell session

A restart was required to enable the Virtual Machine Platform feature. The reported WSL version, **2.7.14.0**, meets the minimum of 2.1.5 specified in the walkthrough.



```
Administrator: Windows PowerShell
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\WINDOWS\system32> wsl --version
WSL version: 2.7.14.0
Kernel version: 6.18.33.2-2
WSLg version: 1.0.73.2
MSRDC version: 1.2.7214
Direct3D version: 1.611.1-81528511
DXCore version: 10.0.26100.1-240331-1435.ge-release
Windows version: 10.0.26200.9457
PS C:\WINDOWS\system32>
```

Figure 3: `wsl --version` confirming WSL 2.7.14.0, above the 2.1.5 minimum

3.2 Docker Desktop Installation

Docker Desktop for Windows was downloaded from the official Docker Desktop release notes page and installed from the downloaded installer.

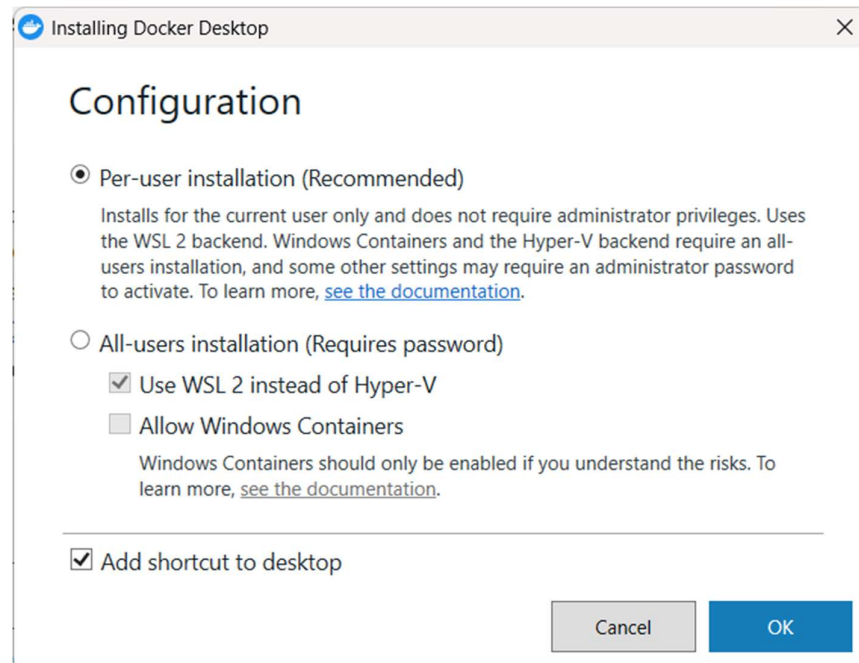


Figure 4: Installer configuration with Per-user installation (WSL 2 backend) selected

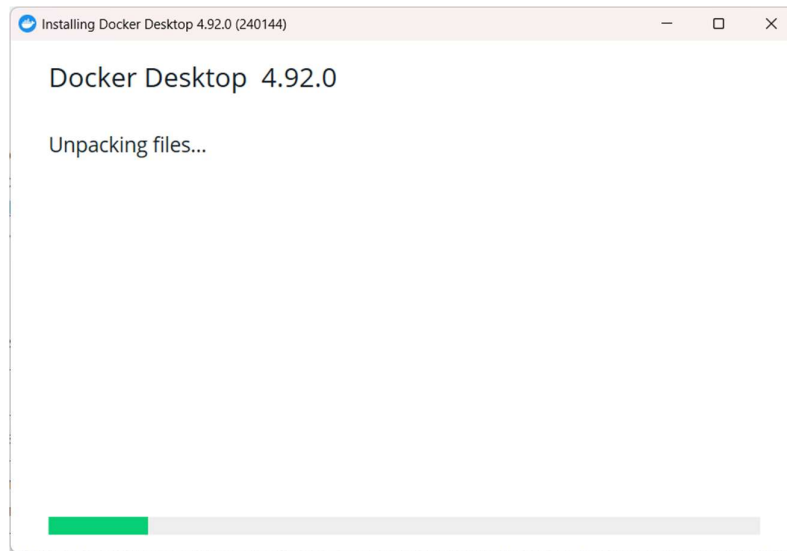


Figure 5: Installer unpacking files

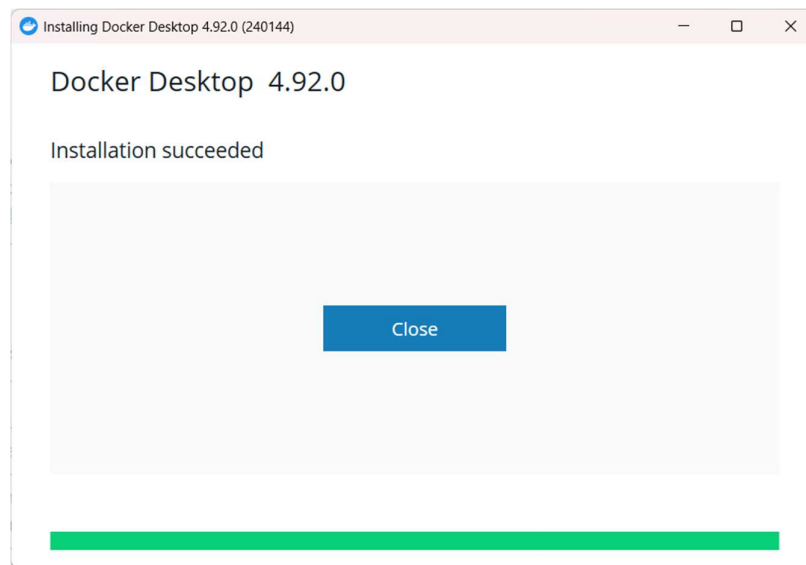


Figure 6: Installation succeeded dialog

The **Per-user installation** option was selected. It installs Docker Desktop for the current account only, needs no administrator privileges and uses the WSL 2 backend. The All-users option is required only for the Hyper-V backend or for Windows containers, neither of which this task uses. Running tooling with the least privilege that works is a principle that recurs throughout the rest of this report.

3.3 Docker Hub Account and Sign-in

On first launch the Docker Subscription Service Agreement was accepted. A Docker Hub account was then created and used to sign in to Docker Desktop, completing the browser-based sign-in handshake. This account owns the public repositories published at Credit, Distinction and High Distinction levels.

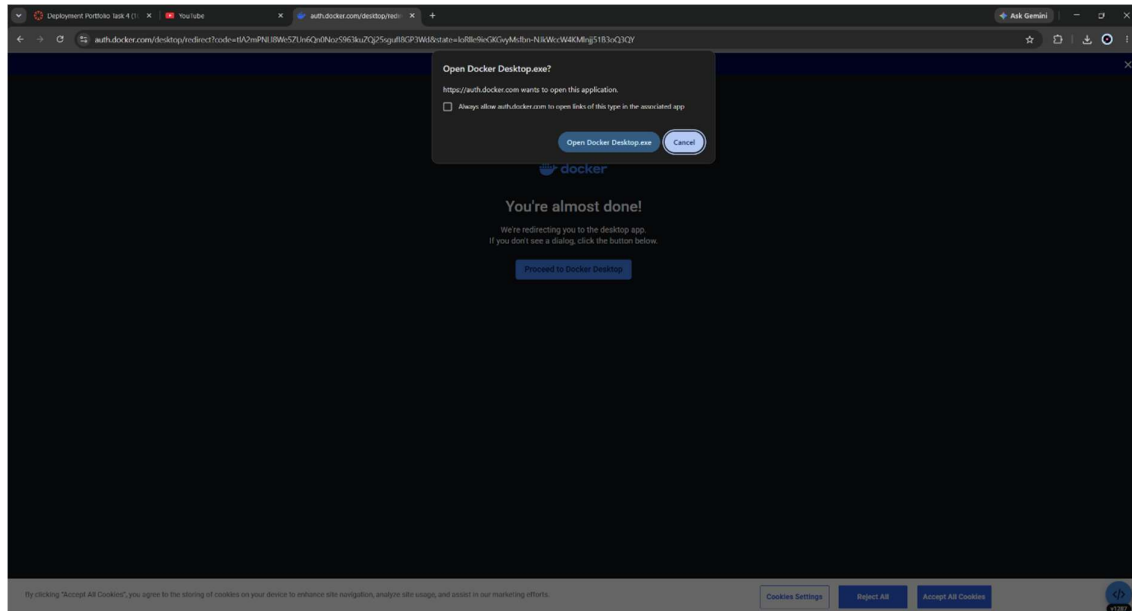


Figure 7: Browser hand-off returning the authenticated session to Docker Desktop

3.4 Verification

```
docker info
docker run hello-world
```

`docker info` confirms that the CLI can reach the Docker daemon: the Server section appears only when the daemon is running, and reports the engine version, the storage driver and the WSL 2 kernel. `docker run hello-world` exercises the full path in one command. The client asks the daemon to run the image; the daemon does not find it locally, pulls it from Docker Hub, creates a container, runs it and streams its output back to the terminal. The container then exits, because its only process has finished. This is expected behaviour, and the first illustration of the container lifecycle examined in depth at Task 4.4.

```

Administrator: Windows PowerShell
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\WINDOWS\system32> docker info
Client:
Version: 29.8.0
Context: desktop-linux
Debug Mode: false
Plugins:
agent: Docker AI Agent Runner (Docker Inc.)
  Version: v1.140.0
  Path: C:\Users\karam\.docker\cli-plugins\docker-agent.exe
ai: Docker AI Agent - Ask Gordon (Docker Inc.)
  Version: v1.31.0
  Path: C:\Users\karam\.docker\cli-plugins\docker-ai.exe
buildx: Docker Buildx (Docker Inc.)
  Version: v0.37.1
  Path: C:\Users\karam\.docker\cli-plugins\docker-buildx.exe
compose: Docker Compose (Docker Inc.)
  Version: v5.5.1
  Path: C:\Users\karam\.docker\cli-plugins\docker-compose.exe
debug: Get a shell into any image or container (Docker Inc.)
  Version: 0.0.47
  Path: C:\Users\karam\.docker\cli-plugins\docker-debug.exe
desktop: Docker Desktop commands (Docker Inc.)
  Version: v0.4.4
  Path: C:\Users\karam\.docker\cli-plugins\docker-desktop.exe
dhi: CLI for managing Docker Hardened Images (Docker Inc.)
  Version: v0.0.7
  Path: C:\Users\karam\.docker\cli-plugins\docker-dhi.exe
extension: Manages Docker extensions (Docker Inc.)
  Version: v0.2.31
  Path: C:\Users\karam\.docker\cli-plugins\docker-extension.exe
init: Creates Docker-related starter files for your project (Docker Inc.)
  Version: v1.4.0
  Path: C:\Users\karam\.docker\cli-plugins\docker-init.exe
mcp: Docker MCP Plugin (Docker Inc.)
  Version: v0.43.3
  Path: C:\Users\karam\.docker\cli-plugins\docker-mcp.exe
model: Docker Model Runner (Docker Inc.)
  Version: v1.2.6
  Path: C:\Users\karam\.docker\cli-plugins\docker-model.exe
offload: Docker Offload (Docker Inc.)
  Version: v0.6.27
  Path: C:\Users\karam\.docker\cli-plugins\docker-offload.exe
pass: Docker Pass Secrets Manager Plugin (beta) (Docker Inc.)
  Version: v0.2.2
  Path: C:\Users\karam\.docker\cli-plugins\docker-pass.exe
sandbox: "docker sandbox" is deprecated, use Docker Sandboxes instead (Docker Inc.)
  Version: v0.13.0
  Path: C:\Users\karam\.docker\cli-plugins\docker-sandbox.exe
scout: Docker Scout (Docker Inc.)
  Version: v1.24.0
  Path: C:\Users\karam\.docker\cli-plugins\docker-scout.exe
Server:
Containers: 0
Running: 0
Paused: 0
Stopped: 0
Images: 0
Server Version: 29.8.0
Storage Driver: overlays
  driver-type: io.containerd.snapshotter.v1
Logging Driver: json-file
Group Driver: groupfs
Cgroup Version: 2

```

Figure 8: docker info output confirming the client can reach the Docker Engine

```

PS C:\WINDOWS\system32> docker run hello-world
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
4f55086f7dd0: Pull complete
d5e71e642bf5: Download complete
Digest: sha256:5e23090353324d887c48ad5e5c56d294eab81588df9605b07d1afe895f9cc8f8
Status: Downloaded newer image for hello-world:latest

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
1. The Docker client contacted the Docker daemon.
2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
   (amd64)
3. The Docker daemon created a new container from that image which runs the
   executable that produces the output you are currently reading.
4. The Docker daemon streamed that output to the Docker client, which sent it
   to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
$ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
https://hub.docker.com/

For more examples and ideas, visit:
https://docs.docker.com/get-started/

PS C:\WINDOWS\system32>

```

Figure 9: docker run hello-world pulling the image and printing "Hello from Docker!"

3.5 Limits of the Pass-Level Result

- It proves the runtime works but deploys nothing of my own: there is no Dockerfile, no custom image and no registry push.
- A single host means portability, the central promise of containers, is untested.

4. Task 4.2 - Credit Level: Containerised Python Application on Two Hosts

Objective: build and containerise a basic Python web application, configure its runtime dependencies, deploy the container listening on a designated host port, push the image to Docker Hub, and pull and run it on a secondary Docker host.

4.1 Source Control and Registry Preparation

All source code, Dockerfiles and scripts for this task live in a public GitHub repository, created and published with GitHub Desktop. The repository's `.gitignore` was in place before the first commit, so the local `.env` configuration file, Python caches and runtime data could never enter the history.

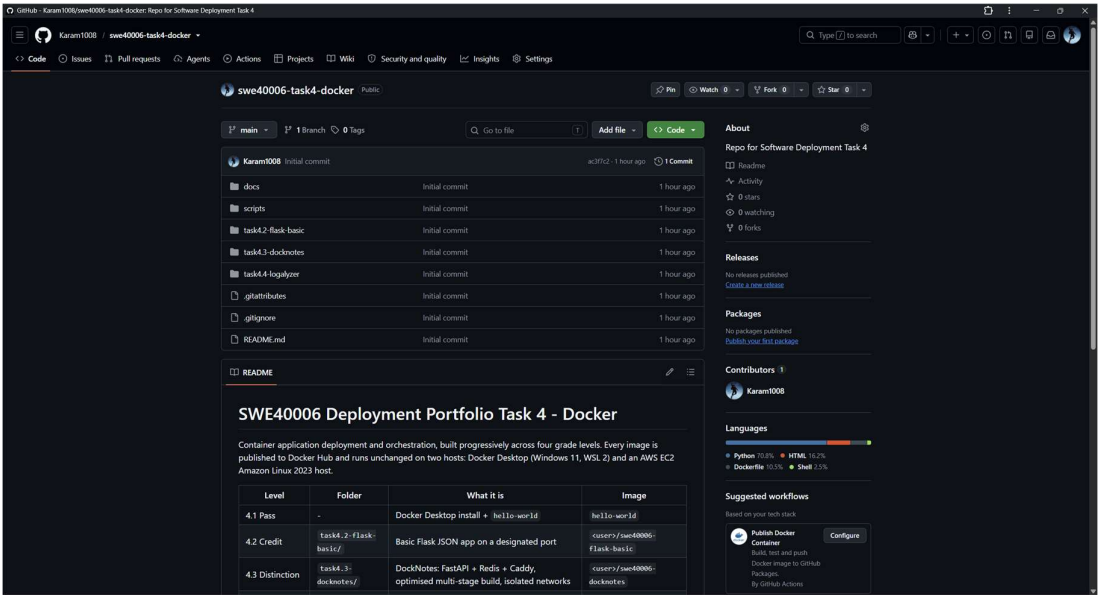


Figure 10: The public GitHub repository showing the per-level folder structure

The Docker CLI was authenticated to Docker Hub with a Personal Access Token rather than the account password. A token can be limited to Read & Write access and revoked on its own without changing the account password, so a leaked token has a smaller blast radius.

```

Administrator: Windows PowerShell
Built: Thu Sep 3 21:53:38 2026
OS/Arch: windows/amd64
Context: desktop-linux

Server: Docker Desktop 4.92.0 (240144)
Engine:
  Version: 29.8.0
  API version: 1.56 (minimum version 1.40)
  Go version: go1.26.8
  Git commit: 3ce5872
  Built: Thu Sep 3 21:51:20 2026
  OS/Arch: linux/amd64
  Experimental: false
containerd:
  Version: v2.3.5
  GitCommit: 1294c24a7da8e5a793ed378161673abe94118892
runc:
  Version: 1.5.1
  GitCommit: v1.5.1-0-g8f2685a4
docker-init:
  Version: 0.19.0
  GitCommit: de40ad0
PS C:\WINDOWS\system32> docker login
Authenticating with existing credentials... [Username: karamjot1008]

Info -> To login with a different account, run 'docker logout' followed by 'docker login'

Login Succeeded
PS C:\WINDOWS\system32>

```

Figure 11: docker version showing client and server, and docker login succeeding

4.2 Application Design

The Credit application is a minimal Flask service with two routes:

Route	Response	Purpose
GET /	JSON: message, docker_host, container_hostname, python_version, started_utc	Identifies which host and which container served the request
GET /health	{"status": "ok"}, HTTP 200	Simple liveness check

The `container_hostname` field exists specifically for this task. Inside a container the hostname defaults to the container ID, so the same image reports a different value on each host. That observable difference proves two separate containers are serving the same code. `docker_host` is read from the `DEPLOY_HOST` environment variable and labels the machine.

```

port = int(os.environ.get("PORT", "5000"))
# 0.0.0.0 is required inside a container; 127.0.0.1 would only accept
# connections from inside the container itself.
app.run(host="0.0.0.0", port=port)

```

Binding to `0.0.0.0` is essential. Flask's default of `127.0.0.1` is the loopback interface of the container's own network namespace, but Docker's port forwarding delivers traffic to the container's `eth0` interface. A loopback-bound server would therefore refuse every forwarded connection while the container still appeared to be running normally.

4.3 Dockerfile

```
FROM python:3.12-slim
WORKDIR /app

# Dependencies first so this layer is cached when only app.py changes
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY app.py .

ENV PORT=5000
EXPOSE 5000
CMD ["python", "app.py"]
```

Instruction	Purpose
FROM python:3.12-slim	Debian-based Python runtime without compilers and documentation
COPY requirements.txt then RUN pip install	Installs the pinned dependency (Flask 3.1.1) in its own layer, reused until requirements.txt changes
--no-cache-dir	Stops pip's download cache being baked into the layer
COPY app.py	Application code is copied last because it changes most often
ENV PORT / EXPOSE 5000	Sets the designated port and documents it. EXPOSE does not publish anything; publishing happens with -p at run time
CMD ["python", "app.py"]	Exec form: Python runs as PID 1 and receives stop signals directly

Instruction order matters because Docker caches each layer and invalidates every layer after the first one that changes. Placing dependency installation before the code copy means a code-only change rebuilds in seconds instead of reinstalling Flask.

4.4 Build

```
cd task4.2-flask-basic
docker build -t swe40006-flask-basic:1.0.0 .
docker images swe40006-flask-basic
docker history swe40006-flask-basic:1.0.0
```

The resulting image is 48.2 MB. `docker history` shows one layer per instruction, confirming the layer ordering described above.

```
PS E:\Swindburne\Software Deployment and Evolution\Deployment Task 4\swed0006-task4-docker\task4.2-flask-basic> docker history swe40006-flask-basic:1.0.0
```

IMAGE	CREATED	CMD	SIZE	COMMENT
01aldb0eebe1	About a minute ago	ENV ["python" "app.py"]	0B	buildkit.dockerfile.v0
c8c95581	About a minute ago	EXPOSE [5000/tcp]	0B	buildkit.dockerfile.v0
c8c95581	About a minute ago	CMD ["PORT" "5000"]	0B	buildkit.dockerfile.v0
c8c95581	About a minute ago	COPY app.py . # buildkit	12.3kB	buildkit.dockerfile.v0
c8c95581	About a minute ago	RUN /bin/sh -c pip install --no-cache-dir -r .	15.3MB	buildkit.dockerfile.v0
c8c95581	About a minute ago	COPY requirements.txt . # buildkit	12.3kB	buildkit.dockerfile.v0
c8c95581	About a minute ago	WORKDIR app	8.19kB	buildkit.dockerfile.v0
c8c95581	8 days ago	CMD ["python3"]	0B	buildkit.dockerfile.v0
c8c95581	8 days ago	RUN /bin/sh -c set -eux; for src in idl3 p...	16.4kB	buildkit.dockerfile.v0
c8c95581	8 days ago	RUN /bin/sh -c set -eux; savedApkFlark="\$a...	41.4MB	buildkit.dockerfile.v0
c8c95581	8 days ago	ENV PYTHON_SHA256=5d8422f579bbaf43a321a559...	0B	buildkit.dockerfile.v0
c8c95581	8 days ago	ENV PYTHON_VERSION=3.12.14	0B	buildkit.dockerfile.v0
c8c95581	8 days ago	ENV GPG_KEY=F16960F62C75135D0D5A26A821E680...	0B	buildkit.dockerfile.v0
c8c95581	8 days ago	RUN /bin/sh -c set -eux; apt-get update; a...	4.95MB	buildkit.dockerfile.v0
c8c95581	8 days ago	ENV LANG=C.UTF-8	0B	buildkit.dockerfile.v0
c8c95581	8 days ago	ENV PATH=/usr/local/bin:/usr/local/sbin:/usr...	0B	buildkit.dockerfile.v0
c8c95581	9 days ago	# debian.sh --arch 'amd64' out 'trixie' @bl...	87.6MB	debuerrreoteo 0.17

```
PS E:\Swindburne\Software Deployment and Evolution\Deployment Task 4\swed0006-task4-docker\task4.2-flask-basic> docker run -d --name flask-basic -p 8080:5000 -- DEPLOY_HOST>windows-laptop swe40006-flask-basic:1.0.0
```

ID	CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
802ab0b008e08	swe40006-flask-basic:1.0.0	"python app.py"	11 seconds ago	Up 10 seconds	0.0.0.0:8080->5000/tcp, :::8080->5000/tcp	flask-basic	

```
PS E:\Swindburne\Software Deployment and Evolution\Deployment Task 4\swed0006-task4-docker\task4.2-flask-basic>
```

4.5 Local Deployment on a Designated Port

```
docker run -d --name flask-basic -p 8080:5000 -e DEPLOY_HOST=windows-laptop swe40006-flask-  
basic:1.0.0  
docker ps  
curl.exe -i http://localhost:8080/health  
docker logs flask-basic
```

`-p 8080:5000` maps host port 8080 to container port 5000. Different numbers were used on purpose, to show that the host port is a deployment decision independent of the port the application listens on. The same image could run several times on one host by mapping different host ports. `-d` runs the container detached, and `-e` injects configuration at run time without changing the image.

```
PS E:\Swinburne\Software Deployment and Evolution\Deployment> Task 4\swe40006-task4-docker\task4.2-flask-basic> docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS                    NAMES
802ab0b08e88   swe40006-flask-basic:1.0.0         "python app.py"         11 seconds ago Up 10 seconds   0.0.0.0:8080->5000/tcp   [:]:8080->5000/tcp      flask-basic
```

Page 17 of 50



Figure 16: Health endpoint returning HTTP 200, and docker logs showing the Flask development-server warning

4.6 Publishing to Docker Hub

The push reports a **digest**, the SHA-256 hash of the image manifest. Unlike a tag, a digest cannot be moved or reused. Any change to any layer produces a different digest, so comparing digests is a conclusive test of whether two hosts hold the same image.

The screenshot shows the Docker Desktop application window. The 'Images' tab is active, displaying a list of Docker images. The 'hello-world' image is selected. The interface includes a search bar, a 'Refresh' button, and a 'Show more images' link. The bottom status bar indicates 'Engine running' and '1.43 GB used (out of 10.6 GB)'.

Name	Tag	Image ID	Created	Size	Actions
hello-world	latest	5c7b91030332	6 months ago	25.87 KB	[Refresh] [Download] [Delete]
kubernetes/venv4000s-flask-basic	1.0.0	61a103bde1	11 minutes ago	197.3 MB	[Refresh] [Download] [Delete]
kubernetes/venv4000s-flask-basic	latest	61a103bde1	11 minutes ago	197.3 MB	[Refresh] [Download] [Delete]
venv4000s-flask-basic	1.0.0	61a103bde1	11 minutes ago	197.3 MB	[Refresh] [Download] [Delete]

The screenshot displays the Docker Hub interface for the repository `karamjpt1008/awef0000e-flask-basic`. The repository is public and was last pushed 3 months ago. The sidebar on the left contains navigation links for various repository features. The main content area shows the repository's general information, including its name, description, and tags. The tags section lists two versions of the image, both labeled `1.0.0`, with their respective push times. The repository overview section at the bottom provides a brief description of the repository's purpose.

Page 19 of 50

4.7 Secondary Docker Host Provisioning

The secondary host is an EC2 instance rather than a second local virtual machine. It is a genuinely separate machine with a different operating system (Amazon Linux 2023 rather than Docker Desktop's WSL 2 virtual machine), a different network and a public address. Distinction level requires public reachability, so the same host serves both levels.

Setting	Value	Rationale
AMI / type	Amazon Linux 2023, t3.micro	Free-tier eligible; SSM Agent preinstalled (as established in Task 3)
Key pair	None	No SSH is needed; administration is through Session Manager
Security group	<code>swe40006-docker-host-sg</code> : TCP 80, 443, 8080 from 0.0.0.0/0	Only the application ports; the launch wizard's default SSH rule was removed
IAM role	<code>swe40006-docker-host-role</code> with <code>AmazonSSMManagedInstanceCore</code> only	Least privilege: the host needs Session Manager and nothing else; Docker Hub pulls of public images need no AWS permissions
User data	<code>scripts/ec2-docker-host-userdata.sh</code>	Installs and starts Docker and the Compose plugin on first boot
Public address	Elastic IP - 52.62.216.119	Stays the same across stop/start, so grading URLs and the HTTPS hostname remain valid

The bootstrap script installs Docker from the Amazon Linux repository and enables the service. It adds `ec2-user` to the `docker` group so that Docker can be used without `sudo`. The Docker Compose v2 plugin is not packaged for Amazon Linux 2023, so the script downloads the official release binary for the architecture reported by `uname -m`. This lets the same script work unchanged on x86 (t3) and Graviton (t4g) instances.

```
dnf install -y docker git
systemctl enable --now docker
usermod -aG docker ec2-user
ARCH="$(uname -m)"
curl -fsSL "https://github.com/docker/compose/releases/latest/download/docker-compose-linux-
${ARCH}" \
-o /usr/local/lib/docker/cli-plugins/docker-compose
```

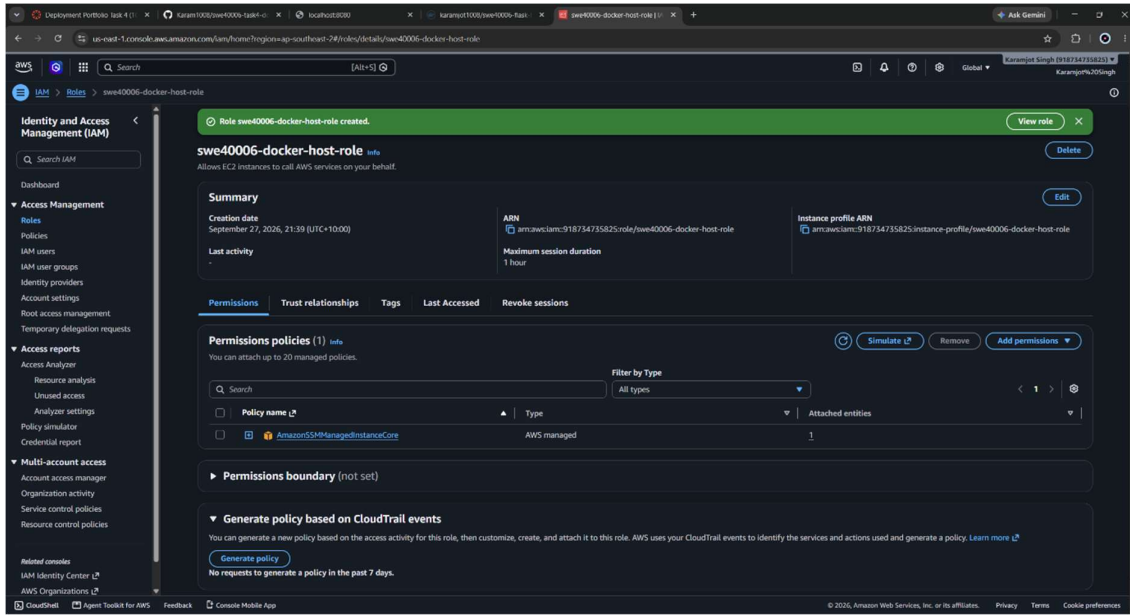


Figure 20: IAM role swe40006-docker-host-role with AmazonSSMManagedInstanceCore as its only policy

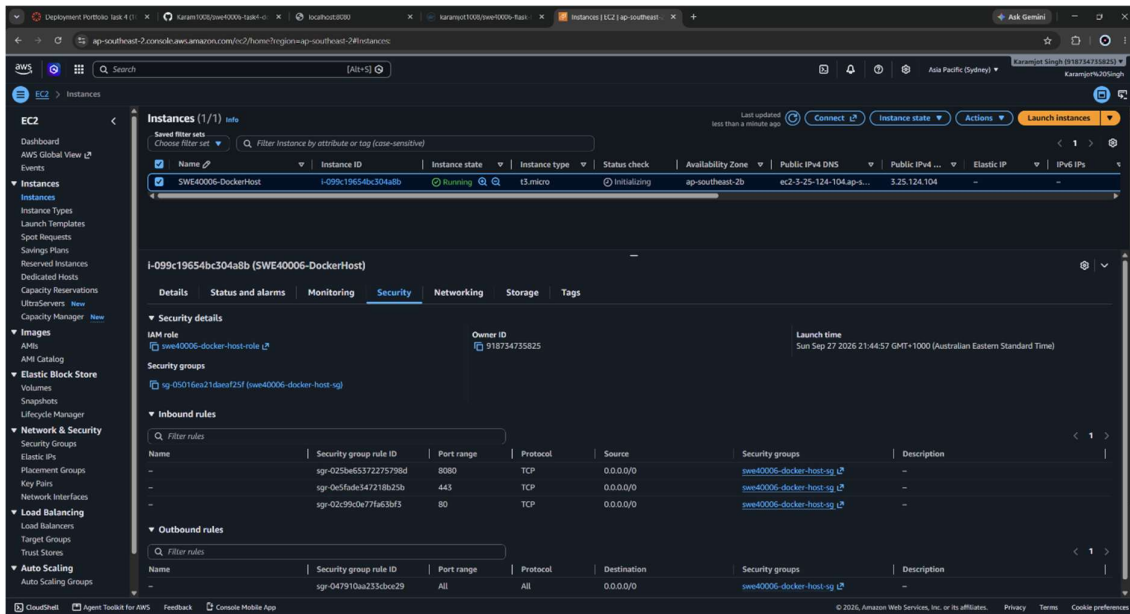


Figure 21: Inbound rules of swe40006-docker-host-sg: 80, 443 and 8080 only, no port 22

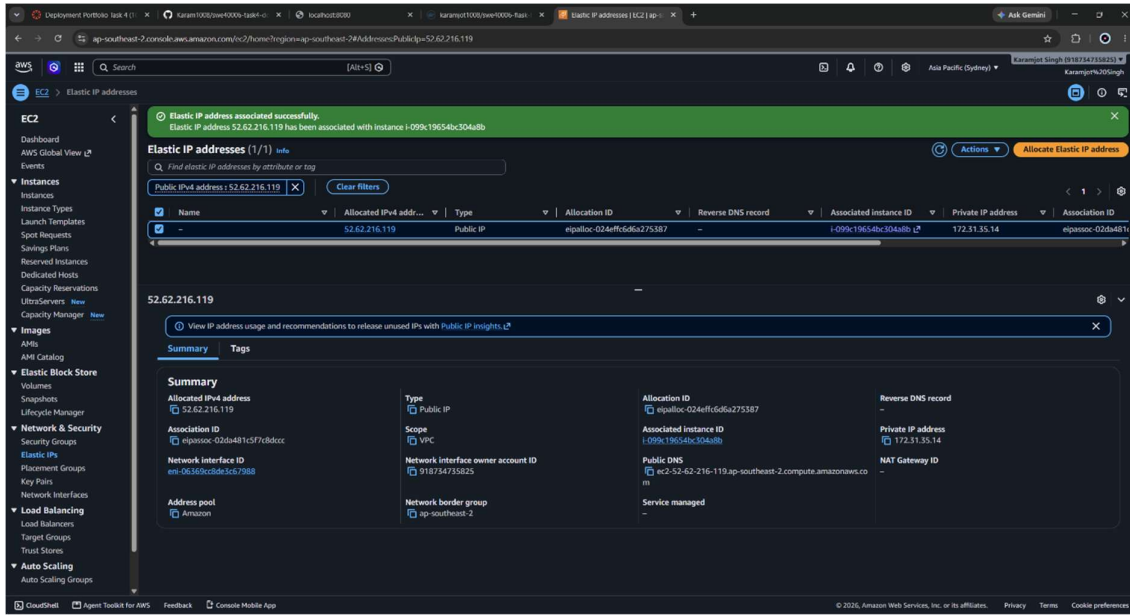


Figure 22: SWE40006-DockerHost details showing the Elastic IP, instance type and IAM role

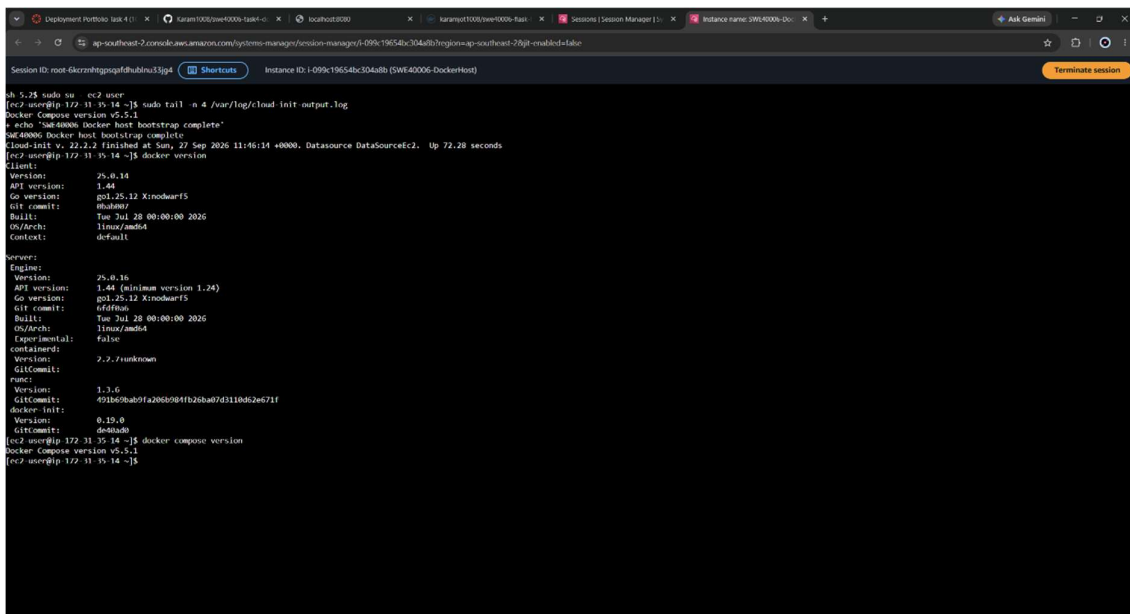


Figure 23: Session Manager session confirming bootstrap completion and the Docker Engine and Compose versions

4.8 Multi-Host Verification

```
# Host 2 - Session Manager, as ec2-user
docker pull karamjot1008/swe40006-flask-basic:1.0.0
docker images --digests karamjot1008/swe40006-flask-basic
docker run -d --name flask-basic --restart unless-stopped \
-p 8080:5000 -e DEPLOY_HOST=aws-ec2 karamjot1008/swe40006-flask-basic:1.0.0
```

The digest on Host 2 is identical to the one recorded on Host 1, which proves the image is byte-for-byte the same artefact rather than a similar rebuild. The browser response then shows `docker_host: aws-ec2` and a container hostname different from the figure above, which proves a separate container on a separate host is serving the same code. `--restart unless-stopped` keeps

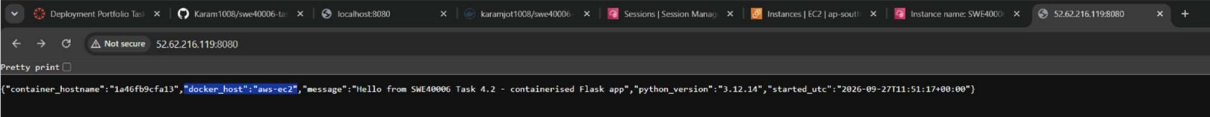
the container running across instance reboots and Docker daemon restarts during the grading period.

```
[ec2-user@ip-172-31-35-14 ~]$ DH=karamjot1008
[ec2-user@ip-172-31-35-14 ~]$ docker pull $DH/swe40006-flask-basic:1.0.0
1.0.0: Pulling from karamjot1008/swe40006-flask-basic
6b37362b3da7: Pull complete
3764a0a7d1e8: Pull complete
b0dc7f87bef1: Pull complete
06ad939ed42b: Pull complete
64ee010963b9: Pull complete
85096d51256d: Pull complete
dbd2f920f950: Pull complete
344ce4c0a2e2: Pull complete
Digest: sha256:61a1d0beebe1c5c9c27b0d28715dfa4ae929d9158fd4d4356b99eded4065955e
Status: Downloaded newer image for karamjot1008/swe40006-flask-basic:1.0.0
docker.io/karamjot1008/swe40006-flask-basic:1.0.0
[ec2-user@ip-172-31-35-14 ~]$ docker images --digests $DH/swe40006-flask-basic
REPOSITORY          TAG          DIGEST
karamjot1008/swe40006-flask-basic  1.0.0       sha256:61a1d0beebe1c5c9c27b0d28715dfa4ae929d9158fd4d4356b99eded4065955e
[ec2-user@ip-172-31-35-14 ~]$
```

Figure 24: docker pull on Host 2 and a digest

```
[ec2-user@ip-172-31-35-14 ~]$ docker ps
CONTAINER ID        IMAGE               COMMAND             CREATED             STATUS              PORTS
1a46fb9cfaf13      karamjot1008/swe40006-flask-basic:1.0.0  "python app.py"     7 seconds ago      Up 6 seconds       0.0.0.0:8080->5000/tcp, :::8080->5000/tcp
[ec2-user@ip-172-31-35-14 ~]$
```

Figure 25: docker ps on Host 2 showing the container running with port 8080 published



```
["container_hostname": "1a46fb9cfaf13", "docker_host": "aws-ec2", "message": "Hello from SWE40006 Task 4.2 - containerised Flask app", "python_version": "3.12.14", "started_utc": "2020-09-27T11:51:17+00:00"]
```

Figure 26: The same image served publicly from Host 2 at <http://52.62.216.119:8080>

4.9 Weaknesses of the Credit-Level Design

The deployment meets the Credit requirements but has five weaknesses, each addressed at Distinction level:

- The process runs as **root** inside the container, so a compromise of the application gives root within the container.
- It uses Flask's **development server**, which is single-process and, as its own warning states, not intended for production.
- There is **no health check**. Docker reports the container as Up as long as the process exists, even if it has stopped answering requests.
- It is a **single stateless container**. A real web application needs state, and state needs storage that outlives the container.
- **Configuration** is two environment variables typed on the command line, with no repeatable definition of how the container should be run.

5. Task 4.3 - Distinction Level: Custom Web Application with an Optimised Build

Objective: develop a new custom web application, write an optimised Dockerfile, deploy it to a containerised environment with exposed network ports, configure container networking and environment variables, and verify public accessibility over HTTP/HTTPS.

5.1 Application Design

DockNotes is a small deployment notes board written with FastAPI. Visitors add short notes, which are stored in Redis together with a visit counter. The page header shows which container served the request, the image version and the configured environment. These values make the container runtime visible in the browser in the same way the Credit application does in JSON.

Route	Behaviour
GET /	Renders notes, the container information panel and the visit counter
POST /notes	Validates the note (at most 280 characters), stores it, then redirects with HTTP 303
GET /api/info	JSON: container hostname, version, environment, Redis reachability, counters
GET /healthz	Liveness probe used by the image's HEALTHCHECK

Three application-level decisions support a reliable deployment:

- **Post/Redirect/Get.** The 303 redirect after a POST means refreshing the page never resubmits a note.
- **Escaped output.** User input is HTML-escaped by Jinja2's autoescaping. A test note containing a `<script>` tag was displayed as plain text, not executed.
- **Graceful degradation.** If Redis is unreachable, the page explains the fault and names the Redis host instead of returning an HTTP 500 error.

5.2 Service Architecture

The application runs as three services defined in `compose.yaml`:

Service	Image	Role	Ports	Networks
proxy	caddy:2-alpine	Reverse proxy, TLS termination, security headers	80, 443 published	frontend
web	Karamjot1008/swe40006-docknotes:1.0.0	FastAPI application under uvicorn (2 workers)	8000 exposed only	frontend, backend
redis	redis:7-alpine	Notes and counter store, append-only persistence	none	backend

A reverse proxy gives the stack one public entry point, similar to the Application Load Balancer in Task 3. Caddy terminates TLS, compresses responses, adds `X-Content-Type-Options`, `X-Frame-Options` and `Referrer-Policy` headers, and removes the `Server` header. This keeps TLS and header

concerns out of the application image, so the image does not change when the certificate or domain does.

5.3 Optimised Dockerfile

```
# ----- Stage 1: builder -----
FROM python:${PYTHON_VERSION}-slim AS builder
ENV PIP_NO_CACHE_DIR=1 PIP_DISABLE_PIP_VERSION_CHECK=1
COPY requirements.txt .
RUN python -m venv /opt/venv && /opt/venv/bin/pip install -r requirements.txt

# ----- Stage 2: runtime -----
FROM python:${PYTHON_VERSION}-slim AS runtime
ENV PYTHONUNBUFFERED=1 PATH="/opt/venv/bin:$PATH"
RUN groupadd --system app && useradd --system --gid app --no-create-home app
COPY --from=builder /opt/venv /opt/venv
COPY app/ ./app/
USER app
EXPOSE 8000
HEALTHCHECK --interval=30s --timeout=3s --start-period=10s --retries=3 \
  CMD ["python", "-c", "import urllib.request;
  urllib.request.urlopen('http://127.0.0.1:8000/healthz', timeout=2)"]
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000", "--workers", "2", "--
  proxy-headers", ...]
```

Technique	Effect
Slim base image	Removes compilers, headers and documentation present in the full <code>python:3.12</code> image (measured in Section 5.4)
Multi-stage build	Dependencies are installed into a virtual environment in the builder stage, and only <code>/opt/venv</code> is copied to the runtime stage. Build tooling and caches never reach the final image
Dependency layer before code	A code change does not reinstall dependencies
<code>PIP_NO_CACHE_DIR</code> , <code>PYTHONDONTWRITEBYTECODE</code>	No pip cache or <code>.pyc</code> files are written into image layers
<code>.dockerignore</code>	Excludes <code>.git</code> , <code>.env</code> , caches and Dockerfiles from the build context, so secrets cannot be copied into a layer by accident
Non-root user <code>app</code>	Fixes the Credit-level weakness: a compromised process has no root privileges in the container
HEALTHCHECK with the Python standard library	Lets Docker detect a hung application without adding curl to the image just for probing
uvicorn with 2 workers, exec-form CMD	Production ASGI server replaces the development server; <code>--proxy-headers</code> preserves the client's scheme and IP behind Caddy; uvicorn is PID 1 and receives SIGTERM
Pinned versions, OCI labels	Reproducible dependency resolution; source repository recorded in image metadata

5.4 Measured Effect of the Optimisation

To quantify the optimisation, the same application was also built from `Dockerfile.naive`, a conventional single-stage file using the full `python:3.12` base and `COPY . .:`

```
docker build -f Dockerfile.naive -t docknotes:naive .
docker compose build
docker images --format "table {{.Repository}}:{{.Tag}}`t{{.Size}}" | Select-String docknotes
```

Image	Dockerfile	Size
docknotes:naive	Single stage, python:3.12	1.65 GB
Karamjot1008/swe40006-docknotes:1.0.0	Multi-stage, python:3.12-slim, non-root	221 MB

A smaller image pulls faster onto Host 2, consumes less registry storage and, more importantly, contains fewer packages that could carry vulnerabilities. The naive image ships a full compiler toolchain that the running application never uses.

```
PS E:\Swinburne\Software Deployment and Evolution\Deployment Task 4\sws40006-task4-docker\task4.3-docknotes> docker images --format "table {{.Repository}};{{.Tag}};{{.Size}}" | Select-String docknotes
Karmats1008/sws40006-docknotes:1.0.0      221MB
docknotes:naive                             1.656B
PS E:\Swinburne\Software Deployment and Evolution\Deployment Task 4\sws40006-task4-docker\task4.3-docknotes>
```

Figure 27: Image size comparison: naive single-stage build versus optimised multi-stage build

[illegible]

Figure 28: docker history of the optimised image showing only runtime layers

Layer caching was demonstrated by adding a comment to `app/main.py` and rebuilding. The dependency installation step was reported as CACHED and the rebuild completed. The comment was then discarded in GitHub Desktop and the image rebuilt, so that the published image matches the repository.

5.5 Local Deployment and Health Checks

```
docker compose up -d
docker compose ps
curl.exe http://localhost/api/info
docker compose logs --tail 15 proxy web
```

Start-up order is driven by health, not just by process start. Redis has a `redis-cli ping` health check. The web image has its own HEALTHCHECK. `depends_on: condition: service_healthy` makes web wait for a healthy Redis, and proxy wait for a healthy web. Without this, Caddy could start forwarding requests before uvicorn was ready, and the first visitors would receive 502 errors.

`/healthz` deliberately returns HTTP 200 even when Redis is down, and reports Redis status in the response body instead. It is a liveness probe, answering whether this container needs restarting. A Redis outage is not fixed by restarting the web container, and treating it as a web failure would add restart churn to an unrelated fault. `restart: unless-stopped` on every service restores the stack after a crash or host reboot.

```
[1] build 1/1
# Image karmaj1100/uae40000-docknotes:1.0.0 Built
# E:\Swishurne\Software Deployment and Evolution\Deployment Task 4\uae40000-task4-docker\task4.3-docknotes> docker compose up -d
[1] up 24/25
# Image caddy:2-alpine pulled
# Image redis:7-alpine pulled
# Network docknotes_backend created
# Volume docknotes_caddy_data created
# Network docknotes_frontend created
# Volume docknotes_redis_data created
# Volume docknotes_caddy_config created
# Container docknotes-redis-1 healthy
# Container docknotes-web-1 healthy
# Container docknotes-proxy-1 Starting
Error response from daemon: ports are not available: exposing port TCP 0.0.0.0:80 -> 127.0.0.1:0: listen tcp 0.0.0.0:80: bind: An attempt was made to access a socket in a way forbidden by its access permissions.

What's next:
Debug this Compose error with Gordon <docker ai "help me fix this compose error">
# E:\Swishurne\Software Deployment and Evolution\Deployment Task 4\uae40000-task4-docker\task4.3-docknotes> docker compose ps
NAME                IMAGE                CPUWARD    SERVICE    CREATED    STATUS    PORTS
docknotes-redis-1   redis:7-alpine       "docker-entrypoint.s..." redis    About a minute ago    Up About a minute (healthy)    6379/tcp
docknotes-web-1     karmaj1100/uae40000-docknotes:1.0.0 "uicorn app.main.m..." web      About a minute ago    Up 53 seconds (healthy)      8000/tcp
[1] up 2/2
# Container docknotes-redis-1 healthy
# Container docknotes-web-1 healthy
# Container docknotes-proxy-1 Starting
Error response from daemon: ports are not available: exposing port TCP 0.0.0.0:80 -> 127.0.0.1:0: listen tcp 0.0.0.0:80: bind: An attempt was made to access a socket in a way forbidden by its access permissions.

What's next:
Debug this Compose error with Gordon <docker ai "help me fix this compose error">
# E:\Swishurne\Software Deployment and Evolution\Deployment Task 4\uae40000-task4-docker\task4.3-docknotes> netstat -ano | findstr ":80"
TCP        0.0.0.0:80          0.0.0.0:0          LISTENING        4
TCP        0.0.0.0:8080        0.0.0.0:0          LISTENING        31208
TCP        292.260.1.195:61490 292.260.1.104:8080 TIME_WAIT        0
TCP        [*]:80             [*]:0              LISTENING        4
TCP        [*]:8080            [*]:0              LISTENING        31208
TCP        [*]:18080           [*]:0              LISTENING        31272
# E:\Swishurne\Software Deployment and Evolution\Deployment Task 4\uae40000-task4-docker\task4.3-docknotes> netsh interface ipv4 show excludedportrange protocol=tcp

Protocol tcp Port Exclusion Ranges

Start Port    End Port
-----
80
27139        27139
30800        30800
56400        56500
56500        56600
56600        56700
56700        56800
56800        56900

* - Administered port exclusions.
# E:\Swishurne\Software Deployment and Evolution\Deployment Task 4\uae40000-task4-docker\task4.3-docknotes> netexec -env
# E:\Swishurne\Software Deployment and Evolution\Deployment Task 4\uae40000-task4-docker\task4.3-docknotes> docker compose up -d
[1] up 3/3
# Container docknotes-redis-1 healthy
# Container docknotes-web-1 healthy
# Container docknotes-proxy-1 Starting
# E:\Swishurne\Software Deployment and Evolution\Deployment Task 4\uae40000-task4-docker\task4.3-docknotes> docker compose ps
NAME                IMAGE                CPUWARD    SERVICE    CREATED    STATUS    PORTS
docknotes-proxy-1   caddy:2-alpine       "caddy run --config ..." proxy    17 seconds ago    Up 16 seconds    0.0.0.0:8081->80/tcp, [::]:8081->80/tcp, 0.0.0.0:8443->443/tcp, [::]:8443->443/tcp
docknotes-redis-1   redis:7-alpine       "docker-entrypoint.s..." redis    5 minutes ago     Up 5 minutes (healthy)    6379/tcp
docknotes-web-1     karmaj1100/uae40000-docknotes:1.0.0 "uicorn app.main.m..." web      5 minutes ago     Up 4 minutes (healthy)    8000/tcp
# E:\Swishurne\Software Deployment and Evolution\Deployment Task 4\uae40000-task4-docker\task4.3-docknotes>
```

Figure 29: docker compose ps: web and redis healthy, only proxy publishing ports

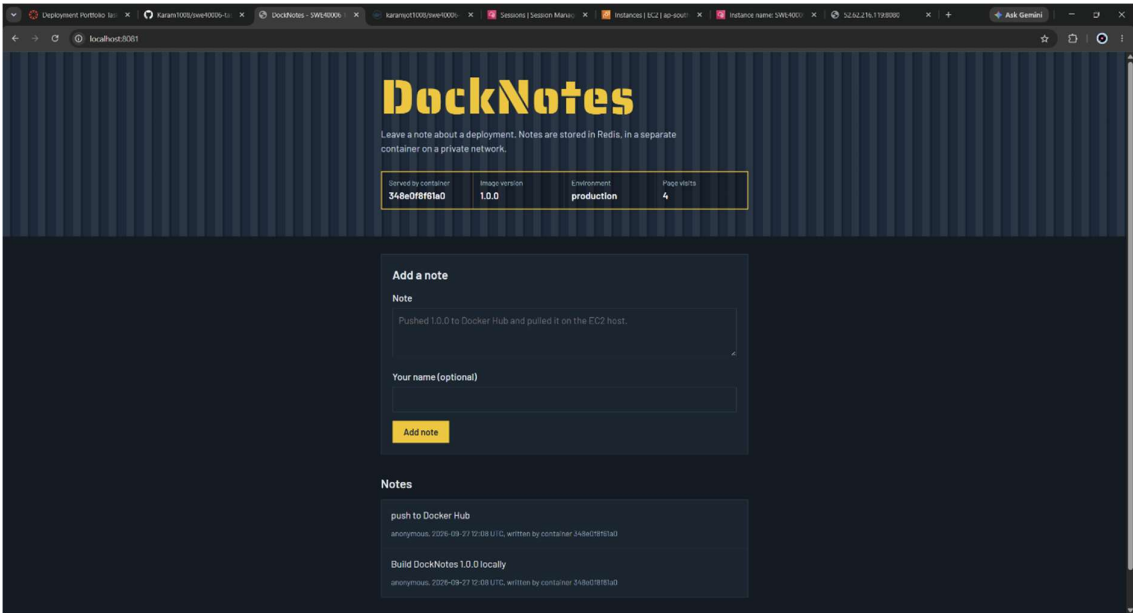


Figure 30: DockNotes at <http://localhost> showing the container information panel and saved notes

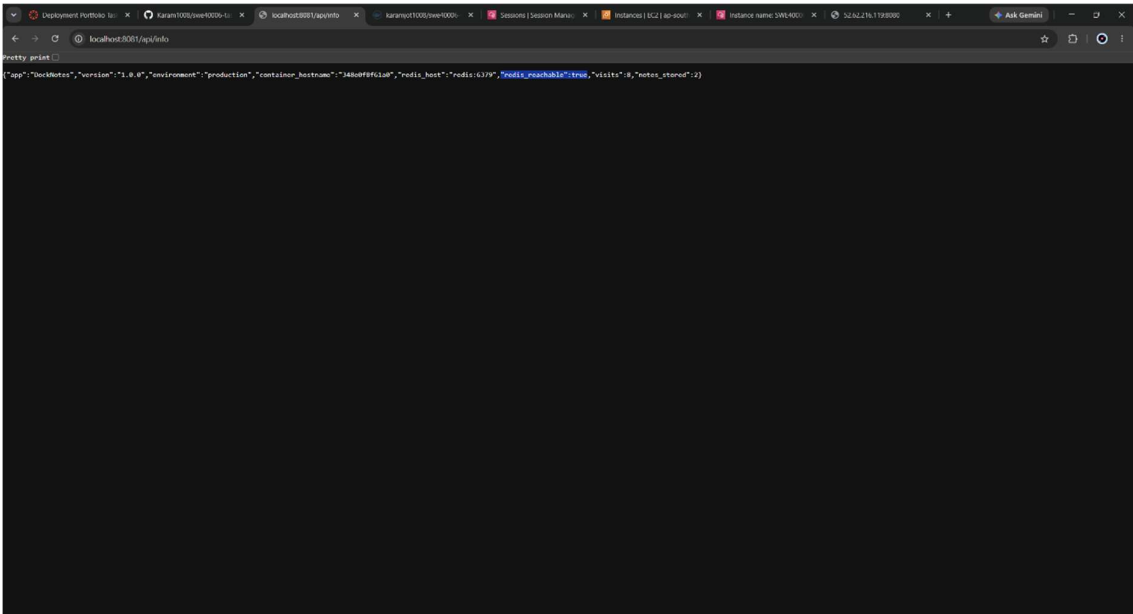


Figure 31: `/api/info` reporting container identity and `redis_reachable: true`

5.6 Container Networking

Network	Internal	Members	Purpose
docknotes_frontend	No	proxy, web	Carries proxied HTTP from Caddy to the application
docknotes_backend	Yes	web, redis	Carries Redis traffic; no route to or from outside Docker

Marking the backend `internal: true` removes its gateway to the outside world. Redis cannot be reached from the host or the internet, and cannot initiate outbound connections. Only the proxy publishes ports; the web service uses `expose`, which documents the port without publishing it. This mirrors the Task 3 design, where each security group admitted traffic only from the tier directly above it.

```
docker network inspect docknotes_backend --format "internal={{.Internal}} containers={{range .Containers}}{{.Name}} {{end}}"
docker compose exec web python -c "import socket; print('redis ->', socket.gethostbyname('redis'))"
docker compose exec proxy wget -q -T 3 -O- http://redis:6379 # expected to fail
curl.exe -m 3 http://localhost:8000 # expected to fail
```

Page 29 of 50

[illegible]

5.7 Environment Variable Configuration

Variable	Source	Effect
DOCKERHUB_USER, APP_VERSION	.env (Compose substitution)	Image name and tag to build, push and pull
APP_ENV, MAX_NOTES	.env, passed into web	Runtime behaviour shown on the page
SITE_ADDRESS	.env, passed into proxy	:80 for HTTP, or a hostname for automatic HTTPS
HTTP_PORT, HTTPS_PORT	.env (port mapping)	Host ports, adjustable if 80/443 are in use
REDIS_HOST, REDIS_PORT	compose.yaml	Service discovery by name on the backend network

Page 30 of 50

To demonstrate this, `APP_ENV` was changed to `staging` and `docker compose up -d` re-run. Compose recreated only the web container, and the page reported the new value without any rebuild. The same step confirmed the process runs as the non-root `app` user.

```
PS E:\Swinburne\Software Deployment and Evolution\Deployment Task 4\swe40006-task4-docker\task4.3-docknotes> docker compose exec web env | Select-String "APP_|REDIS_"
APP_ENV=production
APP_VERSION=1.0.0
REDIS_HOST=redis
REDIS_PORT=6379

PS E:\Swinburne\Software Deployment and Evolution\Deployment Task 4\swe40006-task4-docker\task4.3-docknotes> docker compose exec web whoami
app
PS E:\Swinburne\Software Deployment and Evolution\Deployment Task 4\swe40006-task4-docker\task4.3-docknotes> docker inspect --format "User={{.Config.User}}" "SDH/swe40006-docknotes:1.0.0"
User=app
PS E:\Swinburne\Software Deployment and Evolution\Deployment Task 4\swe40006-task4-docker\task4.3-docknotes>
```

Figure 35: Environment variables inside the web container, and `whoami` returning `app`

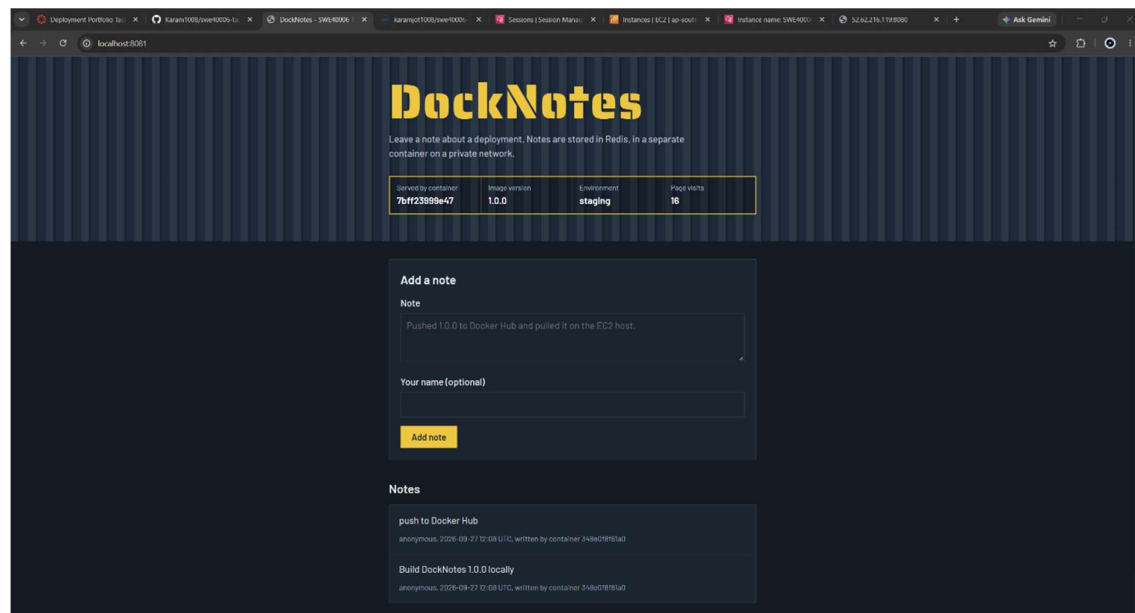


Figure 36: Configuration change applied without rebuilding: the page now reports `staging` and `Notes` still present after `docker compose down` and `up`

5.8 Persistence Across Container Recreation

Redis runs with `--appendonly yes`, writing every change to an append-only file on the `redis_data` named volume. `docker compose down` removes the containers and networks but keeps named volumes unless `-v` is given. After a full down/up cycle, all notes were still present.

5.9 Publishing to Docker Hub

```
docker compose push web
```

Only the web image is custom. The proxy and Redis images are official images that Host 2 pulls directly from Docker Hub. The compressed size shown on Docker Hub, smaller than the local figure because layers are stored compressed.

```
PS E:\Swinburne\Software Deployment and Evolution\Deployment Task 4\swe40006-task4-docker\task4.3-docknotes> docker compose push web
[+] push 5/11
[swe40006-task4-docker\task4.3-docknotes:1.0.0] pushed
PS E:\Swinburne\Software Deployment and Evolution\Deployment Task 4\swe40006-task4-docker\task4.3-docknotes>
```

Figure 37: Docker Hub repository `swe40006-docknotes` with tag `1.0.0`

5.10 Public Deployment on the Secondary Host

```
git clone https://github.com/Karam1008/swe40006-task4-docker.git
cd swe40006-task4-docker/task4.3-docknotes
cp .env.example .env && nano .env      # DOCKERHUB_USER, SITE_ADDRESS=:80
docker compose pull
docker compose up -d --no-build
docker compose ps
```

Host 2 needs only three small files: `compose.yaml`, `Caddyfile` and `.env`. `--no-build` guarantees that every container comes from the registry. If an image were missing, the command would fail instead of quietly building a different one on the server. The public page reports a container hostname different from the one on Host 1, and a note added through the public page was stored and displayed.

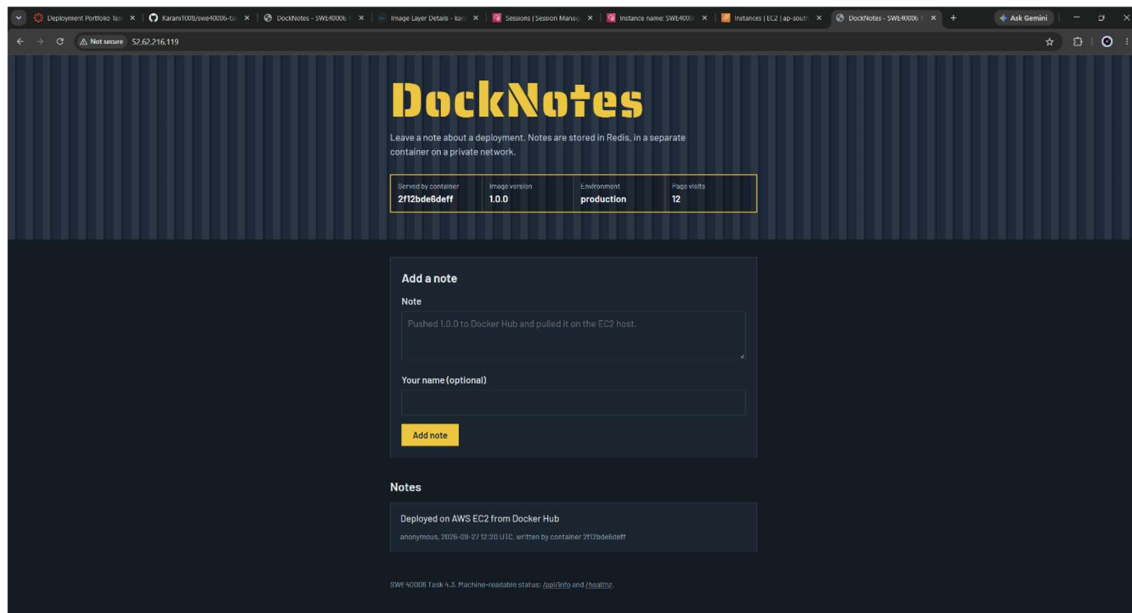


Figure 38: DockNotes served publicly at <http://52.62.216.119>

5.11 HTTPS with Automatic Certificates

Task 3 identified the lack of HTTPS as its most significant weakness, because a certificate requires a domain name that can be validated. That was resolved here without purchasing a domain. `sslip.io` is a public wildcard DNS service that resolves `52.62.216.119.sslip.io` to `52.62.216.119`, giving the Elastic IP a publicly resolvable hostname. With `SITE_ADDRESS` set to that hostname, Caddy's automatic HTTPS obtains a certificate through the ACME protocol, redirects HTTP to HTTPS, and renews the certificate before it expires. Certificates are stored on the `caddy_data` volume, so recreating the proxy container does not request a new certificate and does not exhaust issuance rate limits.

The switch from HTTP to HTTPS was a single environment variable, with no change to the application image. This is the separation of concerns intended when TLS was placed in the proxy rather than in the application.

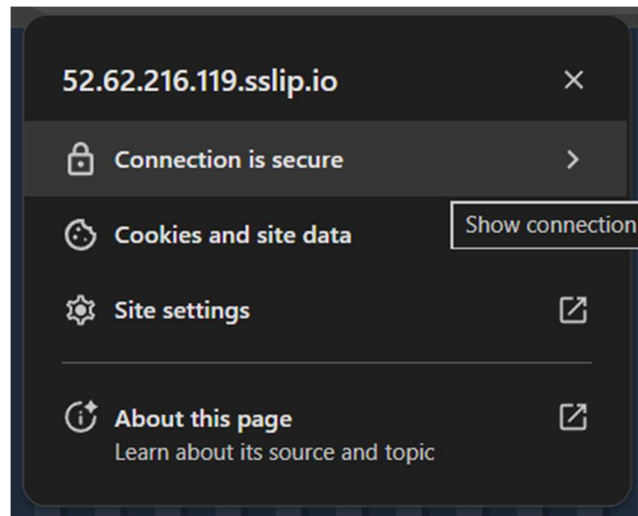


Figure 39: DockNotes served over HTTPS with a valid certificate

6. Task 4.4 - High Distinction Level: Containerised Non-Web Application

Objective: design and implement a simple non-web application, containerise it using Docker best practices, handle persistent volume storage or host mounts, and verify correct container lifecycle execution through logs.

6.1 Tool Design

`logalyzer` is a command-line tool that analyses web server access logs in the Combined Log Format used by Apache, Nginx and most load balancers. This is the log format the web tiers in Task 3 and in this task produce. For each input file it reports request volume, status-code classes, error rate, top paths, top client IPs, the most frequent errors, requests per hour and bytes served. It writes each report as both JSON (for machines) and Markdown (for people). It uses only the Python standard library, so the image installs no third-party packages.

Sub-command	Behaviour
<code>analyze [files]</code>	Analyses the given files, or every <code>*.log</code> in the input directory, then exits
<code>watch --interval N</code>	Long-running: scans the input directory every N seconds and analyses each new file once
<code>history</code>	Lists previous runs recorded on the state volume
<code>generate-sample</code>	Writes a reproducible synthetic access log (seeded) for demonstration

Exit code	Meaning
0	Success
1	Unexpected error, including an unwritable mount
2	Usage error or no input files found
3	An input's error rate exceeded <code>--fail-on-error-rate</code>

Exit codes are the interface a non-web container presents to whatever runs it. A CI pipeline, cron job or scheduler cannot read a report, but it can branch on exit code 3 to raise an alert. Logging is structured `key=value` lines written to stdout. Docker collects a container's stdout and stderr as its logs, so writing log files inside the container would hide them from `docker logs` and lose them when the container is removed.

6.2 Dockerfile and Container Best Practices

```
FROM python:${PYTHON_VERSION}-slim AS base
ENV PYTHONDONTWRITEBYTECODE=1 PYTHONUNBUFFERED=1

FROM base AS test
COPY logalyzer/ ./logalyzer/
COPY tests/ ./tests/
RUN python -m unittest discover -s tests -v

FROM base AS runtime
ARG APP_UID=10001
RUN useradd --system --uid ${APP_UID} --no-create-home --shell /usr/sbin/nologin logalyzer \
  && mkdir -p /data/input /data/output /data/state && chown -R logalyzer:logalyzer /data
COPY logalyzer/ ./logalyzer/ # owned by root: the app cannot modify its own code
ENV INPUT_DIR=/data/input OUTPUT_DIR=/data/output STATE_DIR=/data/state
USER logalyzer
ENTRYPOINT ["python", "-m", "logalyzer"]
CMD ["--help"]
```

Practice	Implementation	Reason
Tests at build time	<code>test</code> stage runs the unit tests	Broken code cannot produce a tested image
Minimal runtime	Slim base; standard library only; no <code>pip install</code>	No third-party supply chain and a small attack surface
Fixed non-root UID	User <code>logalyzer</code> , UID 10001	Predictable ownership for mounted storage
Read-only code	Code copied as root-owned	The running user cannot alter the program
ENTRYPOINT + CMD split	ENTRYPOINT is the tool; CMD is the default argument	The image behaves like an executable: <code>docker run img analyze</code> passes arguments; no arguments shows help
Signal-aware PID 1	Exec form plus explicit SIGTERM/SIGINT handlers	The kernel ignores signals sent to PID 1 unless the process installs a handler; without one, <code>docker stop</code> waits and then kills
Environment configuration	<code>INPUT_DIR</code> , <code>OUTPUT_DIR</code> , <code>STATE_DIR</code> , <code>LOG_LEVEL</code> , <code>WATCH_INTERVAL</code>	Paths and behaviour change without a rebuild
Runtime hardening	<code>--read-only</code> , <code>--tmpfs /tmp</code> , <code>--cap-drop ALL</code> , <code>no-new-privileges</code>	Immutable filesystem and no Linux capabilities
No VOLUME instruction	Storage declared at run time instead	VOLUME silently creates an anonymous volume on every run, leaving orphaned volumes behind

6.3 Tests at Build Time

```
docker build --target test --progress=plain -t logalyzer:test .
```

Six unit tests cover line parsing, query-string stripping, rejection of malformed lines and invalid dates, error-rate arithmetic, the empty-input edge case, and persistence of run history across separate database connections. The last test is a stand-in for a new container attaching to the same

volume. BuildKit builds only the stages a target needs, so tests are run explicitly with `--target test`. In a CI pipeline, this step would gate the build and push.

```

12 resolve image config for docker-image//docker.io/dockerfile:1 ...
13 [auth] docker/dockerfile:pull token for registry-1.docker.io
14 DONE 0.0s
15 [internal] load metadata for docker.io/library/python:3.12-alm
16 ...
17 [auth] library/python:pull token for registry-1.docker.io
18 DONE 0.0s
19 [internal] load metadata for docker.io/library/python:3.12-alm
20 DONE 1.2s
21 [internal] load .dockerignore
22 transferring context: 105B done
23 DONE 0.0s
24 [base 1/2] FROM docker.io/library/python:3.12-11msha256:773bc4da4ee9ef2c70b0053ea08c31f8be030f824190b0ae4d8d6d5628453f
25 resolve docker.io/library/python:3.12-11msha256:773bc4da4ee9ef2c70b0053ea08c31f8be030f824190b0ae4d8d6d5628453f 0.0s done
26 CACHED
27 [base 2/2] MCRDIR /opt/layzlyzer
28 DONE 0.0s
29 [internal] load build context
30 transferring context: 20.33kB done
31 DONE 0.0s
32 [test 1/1] GPPV layzlyzer ././layzlyzer/
33 DONE 0.0s
34 [test 2/1] GPPV tests ././tests/
35 DONE 0.0s
36 [test 1/1] Run python -m unittest discover -s tests -v
37 0.38s test_count_and_error_rate (test_layzlyzer.AnalysisTests.test_count_and_error_rate) ... ok
38 0.38s test_empty_input_has_zero_error_rate (test_layzlyzer.AnalysisTests.test_empty_input_has_zero_error_rate) ... ok
39 0.34s test_history_persist_across_connections (test_layzlyzer.HistoryTests.test_history_persist_across_connections) ... ok
40 0.485 test_dash_size_is_zero (test_layzlyzer.ParamsTests.test_dash_size_is_zero) ... ok
41 0.485 test_garage_combined_format_and_strips_query (test_layzlyzer.ParamsTests.test_garage_combined_format_and_strips_query) ... ok
42 0.485 test_rejects_garbage_and_bad_dates (test_layzlyzer.ParamsTests.test_rejects_garbage_and_bad_dates) ... ok
43 0.485 -----
44 0.485 Run 6 tests in 0.015s
45 0.485
46 0.485 OK
47 DONE 0.0s
48 reporting to image
49 exporting layers 0.1s done
50 exporting manifest sha256:69f886ddc1c757b4c4631881a1333ef4d50b06001837f662c1d7071 done
51 exporting config sha256:50289f216650787996013bd2a29c83ab656cf48841a2d8f3083c4154a done
52 creating manifest list sha256:18831221c8797b61b0e01722a2c4703f0877001036013ab3a20 done
53 exporting manifest list sha256:5092535e75d652ef0d1b32cd811755f1d75163608425a3a2adb7f522d done
54 naming to docker.io/library/python:test done
55 unpacking to docker.io/library/python:test 0.1s done
56 DONE 0.2s
57 [SealOrchestrator/Software Deployment and Evolution/Deployment Task 4]500000-task4-docker/task4-1-layzlyzer)

```

Figure 40: Unit tests running inside the Docker build: Ran 6 tests, OK

6.4 Build and Interface

```
docker build -t karamjot1008/swe40006-logalyzer:1.0.0 .
docker run --rm karamjot1008/swe40006-logalyzer:1.0.0
docker run --rm karamjot1008/swe40006-logalyzer:1.0.0 --version
docker inspect --format "User={{.Config.User}} Entrypoint={{.Config.Entrypoint}}
Cmd={{.Config.Cmd}}" ...
```

The runtime image is **43.3 MB**, essentially the slim Python base plus a few kilobytes of code. Run with no arguments, it prints its help text, which is the default CMD. `docker inspect` confirms the non-root user and the ENTRYPOINT/CMD split.

[illegible]

Figure 41: logalyzer image size

6.5 Storage Design

Container path	Mount type	Mode	Reason
/data/input	Bind mount (host folder)	Read-only	Logs arrive from outside; the tool must never alter its input
/data/output	Bind mount (host folder)	Read-write	Reports must be readable directly on the host
/data/state	Named volume logalyzer_state	Read-write	SQLite run history owned by the container and managed by Docker

Bind mounts suit data that people exchange with the container. Named volumes suit state the container owns: Docker manages their location, they work identically on Windows and Linux hosts, and a new volume is initialised with the ownership of the image directory it is mounted over. Each processed file's SHA-256 hash is recorded in the history database, so watch mode never analyses the same file twice, even across container restarts.

```
docker volume create logalyzer_state
docker run --rm -v "${PWD}\data\input:/data/input" karamjot1008/swe40006-logalyzer:1.0.0
generate-sample --lines 5000
```

6.6 One-Shot Execution in a Hardened Container

```
docker run --rm --name logalyzer-run \
  --read-only --tmpfs /tmp --cap-drop ALL --security-opt no-new-privileges \
  -v "${PWD}\data\input:/data/input:ro" \
  -v "${PWD}\data\output:/data/output" \
  -v logalyzer_state:/data/state \
  karamjot1008/swe40006-logalyzer:1.0.0 analyze
```

The container ran with an immutable root filesystem, no Linux capabilities and no ability to gain privileges, and it still completed normally. This shows the application writes only where storage was deliberately provided. It analysed **5000** lines (**4958** malformed lines skipped and counted) in **91** ms, exited with code 0, and `--rm` removed the container. The reports appeared in the host folder immediately.

```

C:\Users\William\Desktop> docker export --name=sample logzsh
-> exporting to image
-> exporting layers
-> exporting manifest sha256:9f29c55d67aefcc8db8055beaf75ddee20bd4e6113f7b0daadbb631278a
-> saving tar file c:\users\william\desktop\logzsh.tar
-> exporting activation manifest sha256:f9fa9137b083d8d8ee222b0aaedc78f89d238a959511774577b0dadbb3a70
-> exporting manifest list sha256:c8de24ec6dc261cf49ef62041183761cd55d02457c7f1f770bac2a67
-> saving to docker: lo/karajst11080/sample00000-logsampler:1.0.0
-> unpacking to docker: lo/karajst11080/sample00000-logsampler:1.0.0
C:\Users\Software\Deployment and Evolution\Deployment Task 4\sample00000-task-docker\task4-logsampler> docker images kimg
IMAGE                                ID                               DISK USAGE    CONTENT SIZE    EXTRA
sample00000/sample00000-logsampler:1.0.0  e84ef26cd44       179B          43.3KB
C:\Users\Software\Deployment and Evolution\Deployment Task 4\sample00000-task-docker\task4-logsampler> docker run --rm kimg logsampler [-h] ["version"] [analyze,watch,history,generate-sample] ...
Analyze web server access logs (Combined Log Format) and write 350M/Markdown reports.

Optional arguments:
  analyze watch history generate-sample
  analyze      analyze log files once and exit
  watch        poll INPUT_DIR and analyze each new file once
  history      list previous runs
  generate-sample  write a synthetic access log

Options:
  -h, --help            show this help message and exit
  -v, --version          show program's version number and exit

Directory: E:\Users\Software\Deployment and Evolution\Deployment Task 4\sample00000-task-docker\task4-logsampler> docker run --rm kimg --version
logsampler 1.0.0
C:\Users\Software\Deployment and Evolution\Deployment Task 4\sample00000-task-docker\task4-logsampler> docker inspect --format '{{range .Config.Logs}} Entrypoint:{{.Config.Entrypoint}} Cmd:{{.Config.Cmd}}' kimg
logsampler Entrypoint:[python -m logsampler] Cmd:[-help]
C:\Users\Software\Deployment and Evolution\Deployment Task 4\sample00000-task-docker\task4-logsampler> docker volume create logsampler.state
logsampler.state
C:\Users\Software\Deployment and Evolution\Deployment Task 4\sample00000-task-docker\task4-logsampler> docker run --rm /v/[FID]/data/input\data/input? kimg generate-sample --lines 5000
2020-09-27T12:32:17Z level=INFO sample generated path="/data/output/sample-access.log" lines=5000 seed=40000.
C:\Users\Software\Deployment and Evolution\Deployment Task 4\sample00000-task-docker\task4-logsampler> Get-Childitem data/input
Directory: E:\Users\Software\Deployment and Evolution\Deployment Task 4\sample00000-task-docker\task4-logsampler\data/input

Mode                LastWriteTime         length Name
----                -
g-----             27-09-2020   21:05              0 g_kitnet
g-----             27-09-2020   22:32     530111 sample-access.log

PS C:\Users\Software\Deployment and Evolution\Deployment Task 4\sample00000-task-docker\task4-logsampler> docker run --rm sample logsampler-run "
PS > --read-only --tmp /tmp --cap-drop ALL --security-opt no-new-privileges
PS > -v [FID]/data/input:/data/input:ro
PS > -v [FID]/data/output:/data/output:w
PS > -v logsampler.state:/data/state
PS > kimg analyze
PS > echo "tail -n10 cat /dev/urandom"
2020-09-27T12:32:20Z level=INFO analyzed file="sample-access.log" lines=5000 parsed=4938 malformed=42 error_rate=13.10% duration_ms=91 run_id=1
2020-09-27T12:32:20Z level=INFO report written path="/data/output/sample-access-2020092712321000.json"
2020-09-27T12:32:20Z level=INFO report written path="/data/output/sample-access-2020092712321202.md"
2020-09-27T12:32:20Z level=INFO analyze complete files=1
Exit code: 0
PS C:\Users\Software\Deployment and Evolution\Deployment Task 4\sample00000-task-docker\task4-logsampler>

```

Figure 43: Structured log output of the hardened one-shot run, ending with exit code 0

6.7 Exit Codes

With `--fail-on-error-rate 0.05`, the sample's error rate of **13.80%** exceeded the 5% threshold. The tool logged a warning and exited with code 3. When run without an input mount, it found no files and exited with code 2. Both outcomes are distinguishable by any automation that runs the container.

```

Administrator: Windows PowerShell
PS E:\Swinsburne\Software Deployment and Evolution\Deployment Task 4\use40006-task4-docker\task4.4-logger\docker> docker volume create logger_state

logger_state
PS E:\Swinsburne\Software Deployment and Evolution\Deployment Task 4\use40006-task4-docker\task4.4-logger\docker> run --rm -v "%[PND]data\input:/data/input" %$% generate-sample --lines 5000
2026-09-27T12:32:12Z level=INFO sample generated path="/data/input/sample-access.log" lines=5000 seed=40006
PS E:\Swinsburne\Software Deployment and Evolution\Deployment Task 4\use40006-task4-docker\task4.4-logger\docker> Get-Childitem data\input

Directory: E:\Swinsburne\Software Deployment and Evolution\Deployment Task 4\use40006-task4-docker\task4.4-logger\data\input

Mode                LastWriteTime         Length Name
----                -
27-09-2026      21:05                0 .gitkeep
27-09-2026      22:32       500111 sample-access.log

PS E:\Swinsburne\Software Deployment and Evolution\Deployment Task 4\use40006-task4-docker\task4.4-logger\docker> run --rm --name logger-run --
--read-only --tmpfs /tmp --cap-drop ALL --security-opt no-new-privileges --
-v "%[PND]data\input:/data/input:ro" --
-v "%[PND]data\output:/data/output" --
-v logger_state:/data/state
PS E:\Swinsburne\Software Deployment and Evolution\Deployment Task 4\use40006-task4-docker\task4.4-logger\docker> %$% analyze
2026-09-27T12:33:20Z level=INFO analyzed file="sample-access.log" lines=5000 parsed=4958 malformed=42 error_rate=13.80% duration_ms=91 run_id=1
2026-09-27T12:33:20Z level=INFO report written path="/data/output/sample-access-20260927123320Z.json"
2026-09-27T12:33:20Z level=INFO report written path="/data/output/sample-access-20260927123320Z.md"
2026-09-27T12:33:20Z level=INFO analyze complete files=1
exit code: 0
PS E:\Swinsburne\Software Deployment and Evolution\Deployment Task 4\use40006-task4-docker\task4.4-logger\docker> Get-Childitem data\output

Directory: E:\Swinsburne\Software Deployment and Evolution\Deployment Task 4\use40006-task4-docker\task4.4-logger\data\output

Mode                LastWriteTime         Length Name
----                -
27-09-2026      21:05                0 .gitkeep
27-09-2026      22:33       1340 sample-access-20260927123320Z.json
27-09-2026      22:33        801 sample-access-20260927123320Z.md

PS E:\Swinsburne\Software Deployment and Evolution\Deployment Task 4\use40006-task4-docker\task4.4-logger\docker> run --rm -v "%[PND]data\input:/data/input:ro" -v "%[PND]data\output:/data/output" -v logger_state:/data/state %$% analyze --fail-on-error-rate 0.05
2026-09-27T12:33:45Z level=INFO analyzed file="sample-access.log" lines=5000 parsed=4958 malformed=42 error_rate=13.80% duration_ms=88 run_id=2
2026-09-27T12:33:45Z level=INFO report written path="/data/output/sample-access-20260927123345Z.json"
2026-09-27T12:33:45Z level=INFO report written path="/data/output/sample-access-20260927123345Z.md"
2026-09-27T12:33:45Z level=WARNING threshold exceeded error_rate=13.80% limit=5.00%
exit code: 3
What's next:
Debug this container error with Gordon + docker at "help me fix this container error"
exit code: 3
PS E:\Swinsburne\Software Deployment and Evolution\Deployment Task 4\use40006-task4-docker\task4.4-logger\docker> run --rm -v logger_state:/data/state %$% analyze
2026-09-27T12:34:21Z level=ERROR no input files matched dir="/data/input" pattern="*.log"
exit code: 2
What's next:
Debug this container error with Gordon + docker at "help me fix this container error"
exit code: 2
PS E:\Swinsburne\Software Deployment and Evolution\Deployment Task 4\use40006-task4-docker\task4.4-logger\docker> echo "exit code: BLASTOFFIC000"
exit code: 2
PS E:\Swinsburne\Software Deployment and Evolution\Deployment Task 4\use40006-task4-docker\task4.4-logger\docker>

```

Figure 44: Threshold breach returning exit code 3, and missing input returning exit code 2

6.8 State Beyond the Container Lifetime

Every container in the preceding steps was removed on exit, yet `logger_state` history lists all of their runs, each tagged with the hostname of a container that no longer exists. The state belongs to the volume, not to any container, which is the property that allows the tool to be run repeatedly or on a schedule.

```

Administrator: Windows PowerShell
PS E:\Swinsburne\Software Deployment and Evolution\Deployment Task 4\use40006-task4-docker\task4.4-logger\docker> run --rm --name logger-run --
--read-only --tmpfs /tmp --cap-drop ALL --security-opt no-new-privileges --
-v "%[PND]data\input:/data/input:ro" --
-v "%[PND]data\output:/data/output" --
-v logger_state:/data/state
PS E:\Swinsburne\Software Deployment and Evolution\Deployment Task 4\use40006-task4-docker\task4.4-logger\docker> %$% analyze
2026-09-27T12:33:20Z level=INFO analyzed file="sample-access.log" lines=5000 parsed=4958 malformed=42 error_rate=13.80% duration_ms=91 run_id=1
2026-09-27T12:33:20Z level=INFO report written path="/data/output/sample-access-20260927123320Z.json"
2026-09-27T12:33:20Z level=INFO report written path="/data/output/sample-access-20260927123320Z.md"
2026-09-27T12:33:20Z level=INFO analyze complete files=1
exit code: 0
PS E:\Swinsburne\Software Deployment and Evolution\Deployment Task 4\use40006-task4-docker\task4.4-logger\docker> Get-Childitem data\output

Directory: E:\Swinsburne\Software Deployment and Evolution\Deployment Task 4\use40006-task4-docker\task4.4-logger\data\output

Mode                LastWriteTime         Length Name
----                -
27-09-2026      21:05                0 .gitkeep
27-09-2026      22:33       1340 sample-access-20260927123320Z.json
27-09-2026      22:33        801 sample-access-20260927123320Z.md

PS E:\Swinsburne\Software Deployment and Evolution\Deployment Task 4\use40006-task4-docker\task4.4-logger\docker> run --rm -v "%[PND]data\input:/data/input:ro" -v "%[PND]data\output:/data/output" -v logger_state:/data/state %$% analyze --fail-on-error-rate 0.05
2026-09-27T12:33:45Z level=INFO analyzed file="sample-access.log" lines=5000 parsed=4958 malformed=42 error_rate=13.80% duration_ms=88 run_id=2
2026-09-27T12:33:45Z level=INFO report written path="/data/output/sample-access-20260927123345Z.json"
2026-09-27T12:33:45Z level=INFO report written path="/data/output/sample-access-20260927123345Z.md"
2026-09-27T12:33:45Z level=WARNING threshold exceeded error_rate=13.80% limit=5.00%
exit code: 3
What's next:
Debug this container error with Gordon + docker at "help me fix this container error"
exit code: 3
PS E:\Swinsburne\Software Deployment and Evolution\Deployment Task 4\use40006-task4-docker\task4.4-logger\docker> echo "exit code: BLASTOFFIC000"
exit code: 3
PS E:\Swinsburne\Software Deployment and Evolution\Deployment Task 4\use40006-task4-docker\task4.4-logger\docker> run --rm -v logger_state:/data/state %$% analyze
2026-09-27T12:34:21Z level=ERROR no input files matched dir="/data/input" pattern="*.log"
exit code: 2
What's next:
Debug this container error with Gordon + docker at "help me fix this container error"
exit code: 2
PS E:\Swinsburne\Software Deployment and Evolution\Deployment Task 4\use40006-task4-docker\task4.4-logger\docker> echo "exit code: BLASTOFFIC000"
exit code: 2
PS E:\Swinsburne\Software Deployment and Evolution\Deployment Task 4\use40006-task4-docker\task4.4-logger\docker> run --rm -v logger_state:/data/state %$% history
# run_id      source
-----
2026-09-27T12:33:45:00 sample-access.log 5000 42 13.80% 88 028c0b06480
2026-09-27T12:33:20:00 sample-access.log 5000 42 13.80% 91 01eef33a745
PS E:\Swinsburne\Software Deployment and Evolution\Deployment Task 4\use40006-task4-docker\task4.4-logger\docker> docker volume inspect logger_state

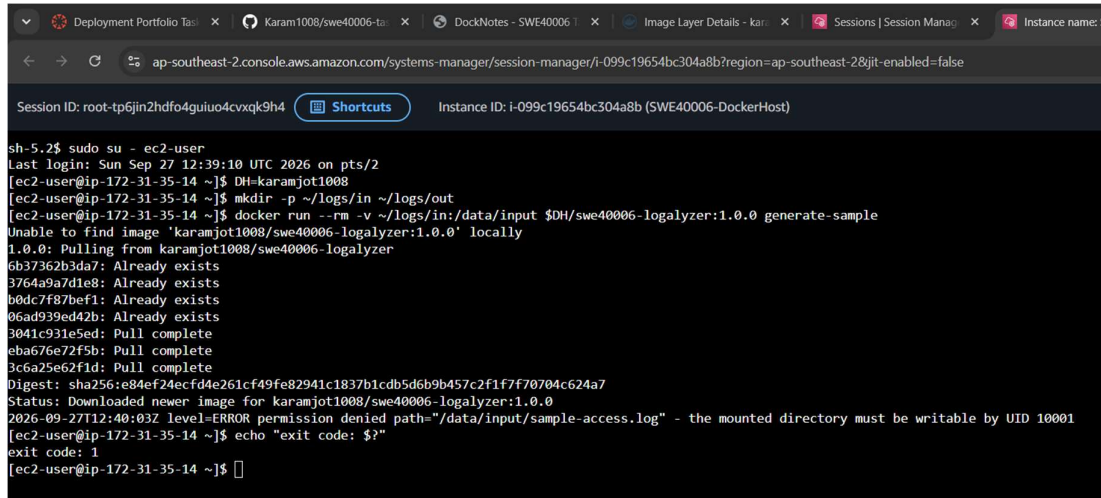
[
  {
    "CreatedAt": "2026-09-27T12:32:12Z",
    "Driver": "local",
    "Labels": null,
    "MountPoint": "/var/lib/docker/volumes/logger_state_data",
    "Name": "logger_state",
    "Options": null,
    "Scope": "local"
  }
]
PS E:\Swinsburne\Software Deployment and Evolution\Deployment Task 4\use40006-task4-docker\task4.4-logger\docker>

```

Figure 45: Run history read from the named volume, recorded by several since-deleted containers

6.10 Portability to the Secondary Host

The image was pushed to Docker Hub and run on Host 2. The first run failed with a permission error that had not occurred on Host 1. After the fix, the same image analysed a log on Amazon Linux and wrote its reports to the host.

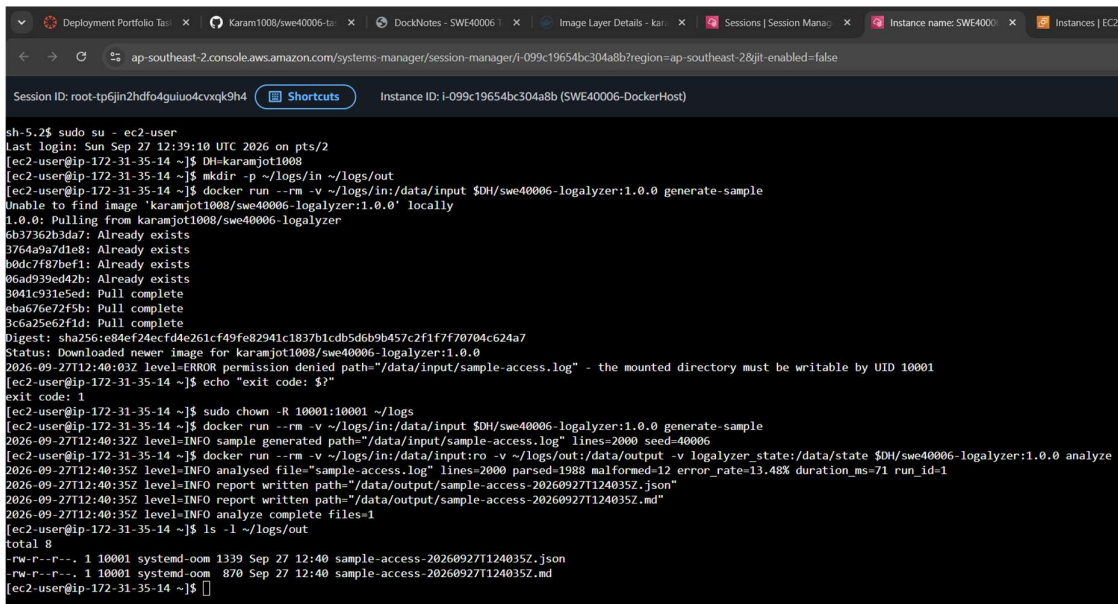


```

sh-5.2$ sudo su - ec2-user
Last login: Sun Sep 27 12:39:10 UTC 2026 on pts/2
[ec2-user@ip-172-31-35-14 ~]$ DH=karamjot1008
[ec2-user@ip-172-31-35-14 ~]$ mkdir -p ~/logs/in ~/logs/out
[ec2-user@ip-172-31-35-14 ~]$ docker run --rm -v ~/logs/in:/data/input $DH/swe40006-logger:1.0.0 generate-sample
Unable to find image 'karamjot1008/swe40006-logger:1.0.0' locally
1.0.0: Pulling from karamjot1008/swe40006-logger
6b37362b3da7: Already exists
3764a9a7d1e8: Already exists
b0dc7f87bef1: Already exists
06ad939ed42b: Already exists
3041c931e5ed: Pull complete
eba676e72f5b: Pull complete
3c6a25e62f1d: Pull complete
Digest: sha256:e84ef24ecfd4e261cf49fe82941c1837b1cdb5d6b9b457c2f1f7f70704c624a7
Status: Downloaded newer image for karamjot1008/swe40006-logger:1.0.0
2026-09-27T12:40:03Z level=ERROR permission denied path="/data/input/sample-access.log" - the mounted directory must be writable by UID 10001
[ec2-user@ip-172-31-35-14 ~]$ echo "exit code: $?"
exit code: 1
[ec2-user@ip-172-31-35-14 ~]$

```

Figure 48: First run on Host 2 failing with permission denied, exit code 1



```

sh-5.2$ sudo su - ec2-user
Last login: Sun Sep 27 12:39:10 UTC 2026 on pts/2
[ec2-user@ip-172-31-35-14 ~]$ DH=karamjot1008
[ec2-user@ip-172-31-35-14 ~]$ mkdir -p ~/logs/in ~/logs/out
[ec2-user@ip-172-31-35-14 ~]$ docker run --rm -v ~/logs/in:/data/input $DH/swe40006-logger:1.0.0 generate-sample
Unable to find image 'karamjot1008/swe40006-logger:1.0.0' locally
1.0.0: Pulling from karamjot1008/swe40006-logger
6b37362b3da7: Already exists
3764a9a7d1e8: Already exists
b0dc7f87bef1: Already exists
06ad939ed42b: Already exists
3041c931e5ed: Pull complete
eba676e72f5b: Pull complete
3c6a25e62f1d: Pull complete
Digest: sha256:e84ef24ecfd4e261cf49fe82941c1837b1cdb5d6b9b457c2f1f7f70704c624a7
Status: Downloaded newer image for karamjot1008/swe40006-logger:1.0.0
2026-09-27T12:40:03Z level=ERROR permission denied path="/data/input/sample-access.log" - the mounted directory must be writable by UID 10001
[ec2-user@ip-172-31-35-14 ~]$ echo "exit code: $?"
exit code: 1
[ec2-user@ip-172-31-35-14 ~]$ sudo chown -R 10001:10001 ~/logs
[ec2-user@ip-172-31-35-14 ~]$ docker run --rm -v ~/logs/in:/data/input $DH/swe40006-logger:1.0.0 generate-sample
2026-09-27T12:40:32Z level=INFO sample generated path="/data/input/sample-access.log" lines=2000 seed=40006
[ec2-user@ip-172-31-35-14 ~]$ docker run --rm -v ~/logs/in:/data/input:ro -v ~/logs/out:/data/output -v logger_state:/data/state $DH/swe40006-logger:1.0.0 analyze
2026-09-27T12:40:35Z level=INFO analysed file="sample-access.log" lines=2000 parsed=1988 malformed=12 error_rate=13.48% duration_ms=71 run_id=1
2026-09-27T12:40:35Z level=INFO report written path="/data/output/sample-access-20260927T124035Z.json"
2026-09-27T12:40:35Z level=INFO report written path="/data/output/sample-access-20260927T124035Z.md"
[ec2-user@ip-172-31-35-14 ~]$ ls -l ~/logs/out
total 8
-rw-r--r-- 1 10001 systemd-oom 1339 Sep 27 12:40 sample-access-20260927T124035Z.json
-rw-r--r-- 1 10001 systemd-oom 870 Sep 27 12:40 sample-access-20260927T124035Z.md
[ec2-user@ip-172-31-35-14 ~]$

```

Figure 49: Successful analysis on Host 2 after correcting bind-mount ownership

6.11 Practice Across the Four Levels

Property	4.1 Pass	4.2 Credit	4.3 Distinction	4.4 High Distinction
Image source	Official image	Custom, single stage	Custom, multi-stage	Custom, with test stage
Process user	n/a	root	app (non-root)	UID 10001 (non-root)
Health / status	Exit only	None	HEALTHCHECK + health-gated start-up	Exit codes + structured logs
Configuration	None	Two ad-hoc env vars	.env with Compose substitution	Env vars + CLI arguments
State	None	None	Named volume (Redis AOF)	Bind mounts + named volume
Network exposure	None	One published port	Proxy only; internal backend	No listener at all
Shutdown	n/a	Default	Exec-form uvicorn	Explicit SIGTERM handler, exit 0

7. Self-Troubleshooting Log and Analysis

Each fault encountered is recorded with the observed symptom, the diagnosis, the resolution and the generalisable lesson. Where an easier but weaker resolution existed, it is stated along with the reason it was rejected.

7.1 Bind-Mount Permission Denied on the Linux Host

```
level=ERROR permission denied path="/data/input/sample-access.log" - the mounted directory must
be writable by UID 10001
exit code: 1
```

Symptom: `generate-sample` failed on Host 2 with exit code 1. The identical command against a bind mount on Host 1 had succeeded .

Diagnosis: The container runs as UID 10001. A bind mount exposes the host directory with its real ownership: `~/logs/in` belonged to `ec2-user` (UID 1000) with mode 775, so UID 10001 fell into the 'other' permission class and had no write access. On Host 1, Docker Desktop mediates Windows folders through its file-sharing layer, which presented the folder as writable and masked the issue. The fault therefore only surfaced on a native Linux engine, which is exactly the kind of difference a second host exists to expose.

Resolution: Ownership of the host folders was given to the container's UID with `sudo chown -R 10001:10001 ~/logs`, and the command re-run successfully.

Rejected alternatives: Running the container with `--user $(id -u):$(id -g)` would fix the bind mount but break the named volume, which Docker initialised from the image as owned by UID 10001, so history writes would then fail instead. Making the directory world-writable (mode 777), or dropping the non-root user, would remove the error by removing the protection.

Lesson: A non-root container makes file ownership part of its interface. Publishing a fixed UID lets the host prepare ownership once, and the application's error message names that UID so the fix is apparent from the log alone.

7.2 Reverse Proxy Restart Loop Caused by an Invalid Site Address

Symptom: After enabling HTTPS, the site refused connections, and docker compose exec reported the proxy container as continuously restarting.

Diagnosis: docker compose logs proxy showed Caddy exiting at start-up with parsing key: invalid port '52.62.216.119.sslip.io': strconv.Atoi: parsing ... invalid syntax. The restart policy then restarted it in a loop. `cat -A .env` showed the value `:52.62.216.119.sslip.io`. A colon left over from the original `:80` value made Caddy treat the whole hostname as a port number. The container logs, not the browser, identified the fault: the browser only reported a refused connection.

Resolution: The value was corrected to `SITE_ADDRESS=52-62-216-119.sslip.io` and the proxy was recreated. Caddy parsed the address, obtained a certificate and began serving HTTPS

Lesson: A restart policy keeps a failing container alive but hides the failure behind a "Restarting" status. When a container restarts repeatedly, read its logs first, because the process's own start-up error is the direct evidence of the cause.

7.3 Host Port 80 Unavailable on the Windows Host

Symptom: docker compose up -d started redis and web, but proxy failed with ports are not available: exposing port TCP 0.0.0.0:80 ... bind: An attempt was made to access a socket in a way forbidden by its access permissions.

Diagnosis: The containers themselves were healthy. The failure happened when Docker tried to claim host port 80 on Windows.

Resolution: Only the host side of the port mapping was changed, using HTTP_PORT=8081 and HTTPS_PORT=8443 in .env. The container still listens on 80, and no Compose file or image change was needed. The public deployment on Host 2 keeps ports 80 and 443.

Rejected alternative: Stopping the Windows service or changing the reserved ranges would free the port, but it changes host operating-system configuration just to suit one development container.

Lesson: Making the host port an environment variable separates a machine-specific detail from the deployment definition. The same compose.yaml runs on a laptop with port 80 blocked and on a server that needs port 80.

7.4 HTTPS URL refused connections.

Symptom: the HTTPS URL refused connections.

Stage 1: docker compose up -d showed the proxy as "Started" rather than "Recreated", which meant the configuration Compose saw hadn't changed.

Stage 2: after recreating the proxy, it went into a restart loop. The logs showed invalid port '52.62.216.119.sslip.io', and cat -A .env revealed a leftover leading colon.

Fix: SITE_ADDRESS=52.62.216.119.sslip.io and recreate the proxy.

Lesson: check the value inside the running container, not the file, and read the logs when a container keeps restarting.

8. Limitations and Future Work

The following limitations are known and were accepted within the scope and cost constraints of this task. They are stated explicitly because an accurate account of what a deployment does not do is as informative as an account of what it does.

8.1 Single-Host Orchestration

Docker Compose orchestrates containers on one host. If [SWE40006-DockerHost](#) fails, the application is offline: nothing reschedules containers elsewhere, unlike the Auto Scaling Group in Task 3. A multi-host scheduler such as Docker Swarm, Kubernetes or Amazon ECS would place replicas across Availability Zones behind a load balancer. The images, health checks and environment-driven configuration built here would carry over unchanged.

8.2 Stateful Service Without Replication or Backup

Redis data lives on a single named volume on a single EBS volume, with no replica and no backup. The loss of the instance loses the notes. A managed service such as Amazon ElastiCache, or a scheduled copy of the volume to S3 following the Task 3 backup pattern, would address this.

8.3 Manual Build and Publish

Images are built and pushed by hand from the laptop, with no CI/CD. A GitHub Actions workflow could run the test stage, build multi-architecture images (amd64 and arm64) with Buildx, and push a versioned tag on each release. This would remove the laptop from the release path entirely.

8.4 No Image Scanning, Signing or Digest Pinning

Images are not scanned for known vulnerabilities or cryptographically signed, and base images are referenced by tag rather than by digest, so a rebuild could pick up a different base image. Docker Scout or Trivy scanning, cosign signing, and digest-pinned base images kept current by an automated dependency bot would close these gaps.

8.5 HTTPS Relies on a Shared Public Domain

HTTPS depends on sslip.io, a third-party DNS service whose shared domain is subject to certificate authority rate limits. A production deployment would use its own domain.

8.6 Registry Credentials and Private Images

All images are public, so Host 2 needs no registry credentials. Private images would require either a Docker Hub pull token stored on the host or migration to Amazon ECR, where the instance role could grant pull access without any stored secret.

8.7 Scope of the Log Analysis Tool

logalyzer understands the Combined Log Format only, and analyses each file in a single pass in memory. That is appropriate for files of megabytes, not gigabytes, and it has no streaming input. A production log pipeline would ship logs to a central service rather than analysing files on a host.

9. External Code and Script Repository

In accordance with the submission guidelines, no full source files, credentials or keys are reproduced in this report. All source code, Dockerfiles, Compose files and scripts are published in a public version-controlled repository: <https://github.com/Karam1008/swe40006-task4-docker>

Path	Purpose
README.md	Repository overview and quick start
task4.2-flask-basic/	Credit: <code>app.py</code> , <code>requirements.txt</code> , <code>Dockerfile</code> , <code>.dockerignore</code>
task4.3-docknotes/app/	Distinction: FastAPI application and HTML template
task4.3-docknotes/Dockerfile	Optimised multi-stage Dockerfile
task4.3-docknotes/Dockerfile.naive	Unoptimised baseline used only for the size comparison
task4.3-docknotes/compose.yaml, Caddyfile, .env.example	Three-service stack, reverse proxy configuration, configuration template
task4.4-logalyzer/logalyzer/	HD: CLI source (analysis, history, command-line interface)
task4.4-logalyzer/tests/	Unit tests executed in the Docker test stage
task4.4-logalyzer/Dockerfile, compose.yaml	Test and runtime stages; watcher service definition
scripts/ec2-docker-host-userdata.sh	Secondary Docker host bootstrap
docs/RUNBOOK.md	Step-by-step execution procedure

No credential is present in the repository. `.env` files are excluded by `.gitignore` and `.dockerignore`, and the Docker Hub token exists only in the local Docker credential store.

10. Generative AI Declaration

This declaration is provided in accordance with Swinburne University's policy on the use of generative artificial intelligence in assessment.

10.1 Tool Used and Scope of Use

A generative AI assistant (Anthropic Claude) was used during this task in three ways:

- **Code generation:** The assistant generated the initial versions of the three applications (the Flask application, the DockNotes FastAPI application and its HTML template, and the logalyzer CLI with its unit tests), together with the Dockerfiles, Compose files, Caddy configuration and the EC2 bootstrap script.
- **Error feedback:** When a command failed, I gave the error output to the assistant for feedback on which layer the fault was likely in. Every fault in Section 7 was confirmed against the actual system state, using the commands shown there, before any change was made.

10.2 What Was Performed by Me

The assistant had no access to my machine, my Docker Hub account, my GitHub account or my AWS account, and could not run any command. I personally:

- Installed WSL 2 and Docker Desktop, created the Docker Hub account and completed Task 4.1.
- Created and published the GitHub repository using GitHub Desktop.
- Ran every build, run, tag, push and pull command recorded in this report, on both hosts.
- Created the IAM role, security group, EC2 instance and Elastic IP, and deployed the containers on Host 2.
- Captured every screenshot and recorded every measured value (image sizes, digests, timings).
- Diagnosed and resolved the faults recorded in Section 7.

10.3 Verification and Independent Judgement

Generated code and guidance were treated as a starting point to be verified against the running system rather than as authoritative:

- The claim that both hosts ran the same artefact was verified by comparing image digests, not assumed from matching tags.
- Network isolation was verified by tests designed to fail, not inferred from the Compose file.
- Every exit code and the SIGTERM handling were observed directly in the container logs and in `docker inspect`.

10.4 Statement

I understand the architecture described in this report, the purpose of every Dockerfile instruction and Compose setting used, and the reasoning behind each design decision recorded. All deployment steps were executed, tested and verified by me. I accept full responsibility for the accuracy of the content of this submission.

Field	Detail
Signed	Karamjot Singh
Student ID	104809887
Date	27/09/2026

11. References

- [1] Microsoft, "Install WSL," Microsoft Learn, 2026. [Online]. Available: <https://learn.microsoft.com/en-us/windows/wsl/install>. [Accessed: Sep. 2026].
- [2] Docker Inc., "Install Docker Desktop on Windows," Docker Docs, 2026. [Online]. Available: <https://docs.docker.com/desktop/setup/install/windows-install/>. [Accessed: Sep. 2026].
- [3] Docker Inc., "Dockerfile reference," Docker Docs, 2026. [Online]. Available: <https://docs.docker.com/reference/dockerfile/>. [Accessed: Sep. 2026].
- [4] Docker Inc., "Building best practices," Docker Docs, 2026. [Online]. Available: <https://docs.docker.com/build/building/best-practices/>. [Accessed: Sep. 2026].
- [5] Docker Inc., "Multi-stage builds," Docker Docs, 2026. [Online]. Available: <https://docs.docker.com/build/building/multi-stage/>. [Accessed: Sep. 2026].
- [6] Docker Inc., "Compose file reference," Docker Docs, 2026. [Online]. Available: <https://docs.docker.com/reference/compose-file/>. [Accessed: Sep. 2026].
- [7] Docker Inc., "Bridge network driver," Docker Docs, 2026. [Online]. Available: <https://docs.docker.com/engine/network/drivers/bridge/>. [Accessed: Sep. 2026].
- [8] Docker Inc., "Volumes," Docker Docs, 2026. [Online]. Available: <https://docs.docker.com/engine/storage/volumes/>. [Accessed: Sep. 2026].
- [9] Docker Inc., "Bind mounts," Docker Docs, 2026. [Online]. Available: <https://docs.docker.com/engine/storage/bind-mounts/>. [Accessed: Sep. 2026].
- [10] Docker Inc., "Start containers automatically," Docker Docs, 2026. [Online]. Available: <https://docs.docker.com/engine/containers/start-containers-automatically/>. [Accessed: Sep. 2026].
- [11] Docker Inc., "Repositories," Docker Hub documentation, 2026. [Online]. Available: <https://docs.docker.com/docker-hub/repos/>. [Accessed: Sep. 2026].
- [12] Docker Inc., "Personal access tokens," Docker Docs, 2026. [Online]. Available: <https://docs.docker.com/security/for-developers/access-tokens/>. [Accessed: Sep. 2026].
- [13] Caddy, "Automatic HTTPS," Caddy Documentation, 2026. [Online]. Available: <https://caddyserver.com/docs/automatic-https>. [Accessed: Sep. 2026].
- [14] Caddy, "reverse_proxy (Caddyfile directive)," Caddy Documentation, 2026. [Online]. Available: https://caddyserver.com/docs/caddyfile/directives/reverse_proxy. [Accessed: Sep. 2026].
- [15] Redis Ltd., "Redis persistence," Redis Documentation, 2026. [Online]. Available: https://redis.io/docs/latest/operate/oss_and_stack/management/persistence/. [Accessed: Sep. 2026].
- [16] S. Ramírez, "FastAPI in Containers - Docker," FastAPI Documentation, 2026. [Online]. Available: <https://fastapi.tiangolo.com/deployment/docker/>. [Accessed: Sep. 2026].
- [17] Pallets, "Deploying to Production," Flask Documentation, 2026. [Online]. Available: <https://flask.palletsprojects.com/en/stable/deploying/>. [Accessed: Sep. 2026].
- [18] Amazon Web Services, "AWS Systems Manager Session Manager," AWS Systems Manager User Guide, 2026. [Online]. Available: <https://docs.aws.amazon.com/systems-manager/latest/userguide/session-manager.html>. [Accessed: Sep. 2026].
- [19] Amazon Web Services, "Elastic IP addresses," Amazon EC2 User Guide, 2026. [Online]. Available: <https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/elastic-ip-addresses-eip.html>. [Accessed: Sep. 2026].

[20] Python Software Foundation, “signal - Set handlers for asynchronous events,” Python 3 Documentation, 2026. [Online]. Available: <https://docs.python.org/3/library/signal.html>. [Accessed: Sep. 2026].

[21] Open Container Initiative, “Pre-Defined Annotation Keys,” OCI Image Format Specification, 2026. [Online]. Available: <https://github.com/opencontainers/image-spec/blob/main/annotations.md>. [Accessed: Sep. 2026].

[22] sslip.io, “sslip.io: DNS for any IP address,” 2026. [Online]. Available: <https://sslip.io/>. [Accessed: Sep. 2026].